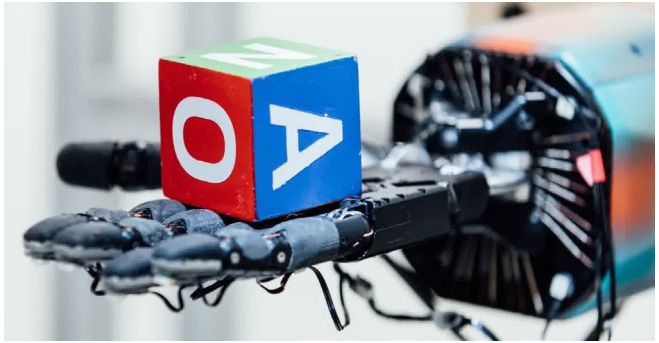
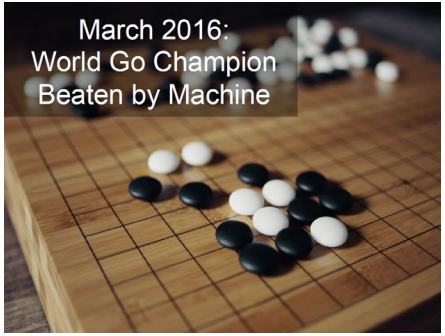
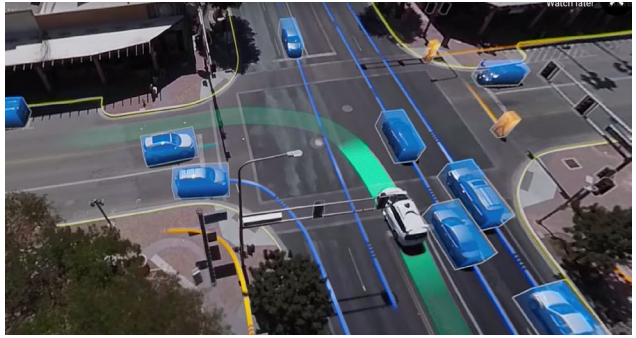
强化学习
Reinforcement Learning







同济大学 TONGJI UNIVERSITY

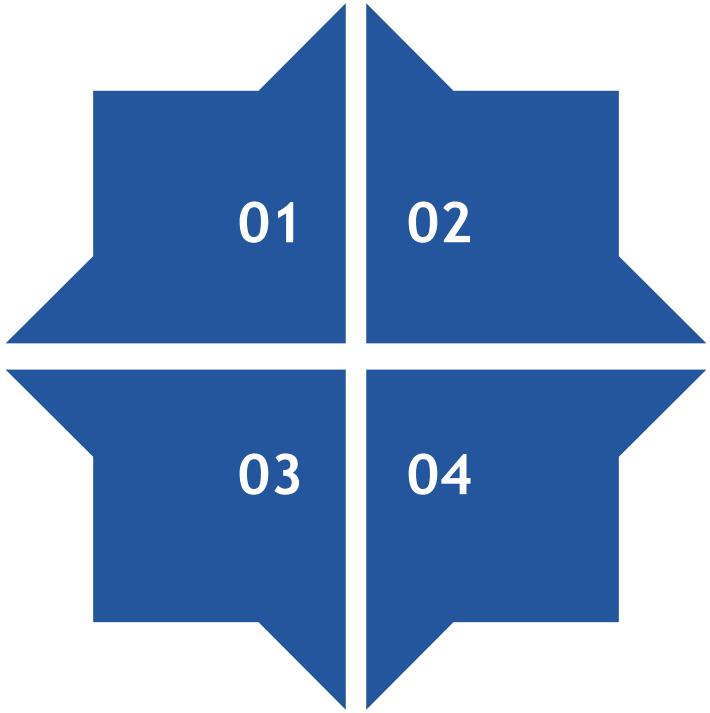
目录

强化学习概述

强化学习的基本概念

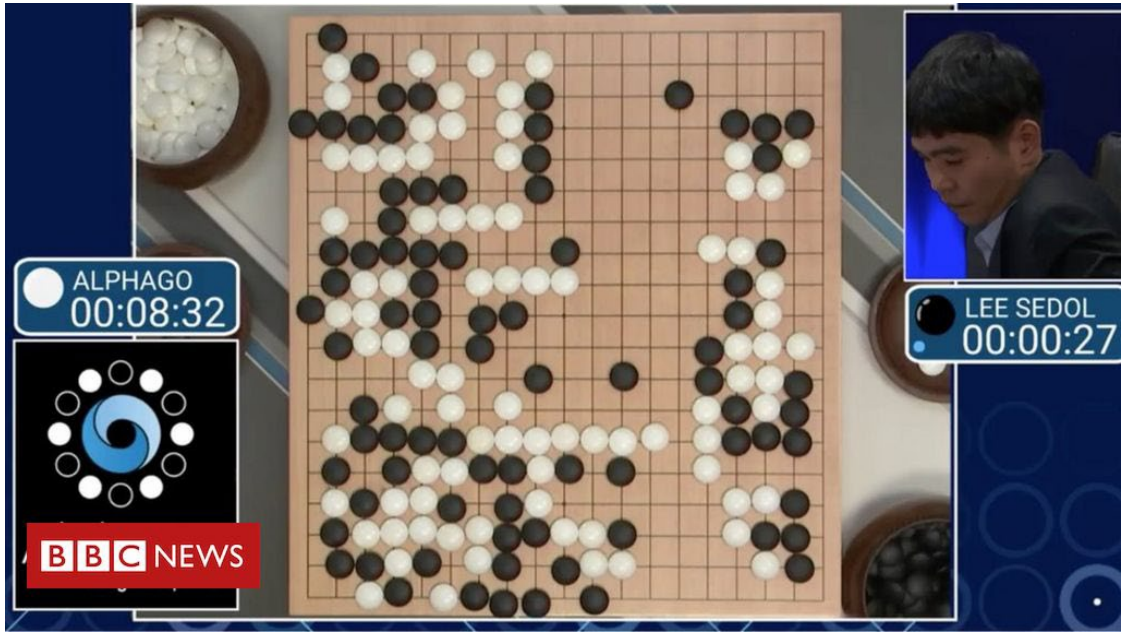
强化学习的数学基础

强化学习的定义



强化学习概述

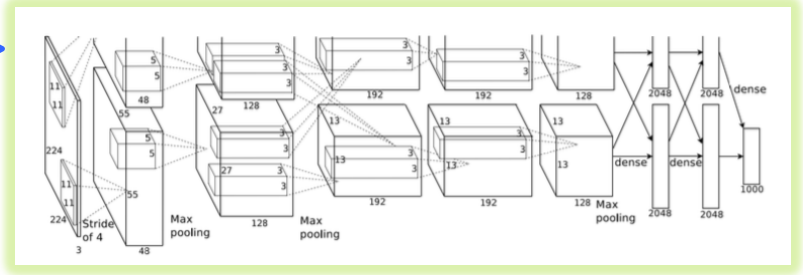
AlphaGo是基于概率计算的，它只是一台机器。但当我看到这个动作时，我改变了主意。
当然，AlphaGo很有创造力。 ——李世石（18个世界围棋冠军）



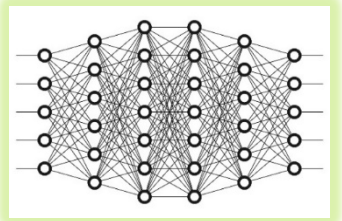
强化学习概述

感知





所有合法移动的概率



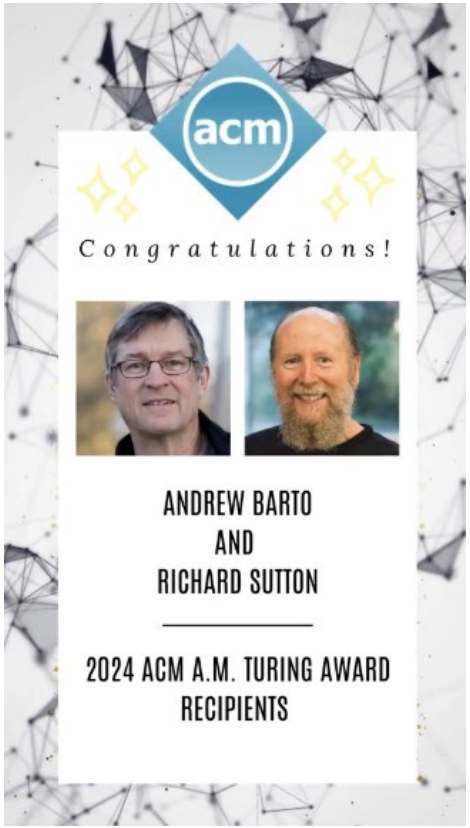
行动
(以最高分移动)

决策与行动

强化学习概述



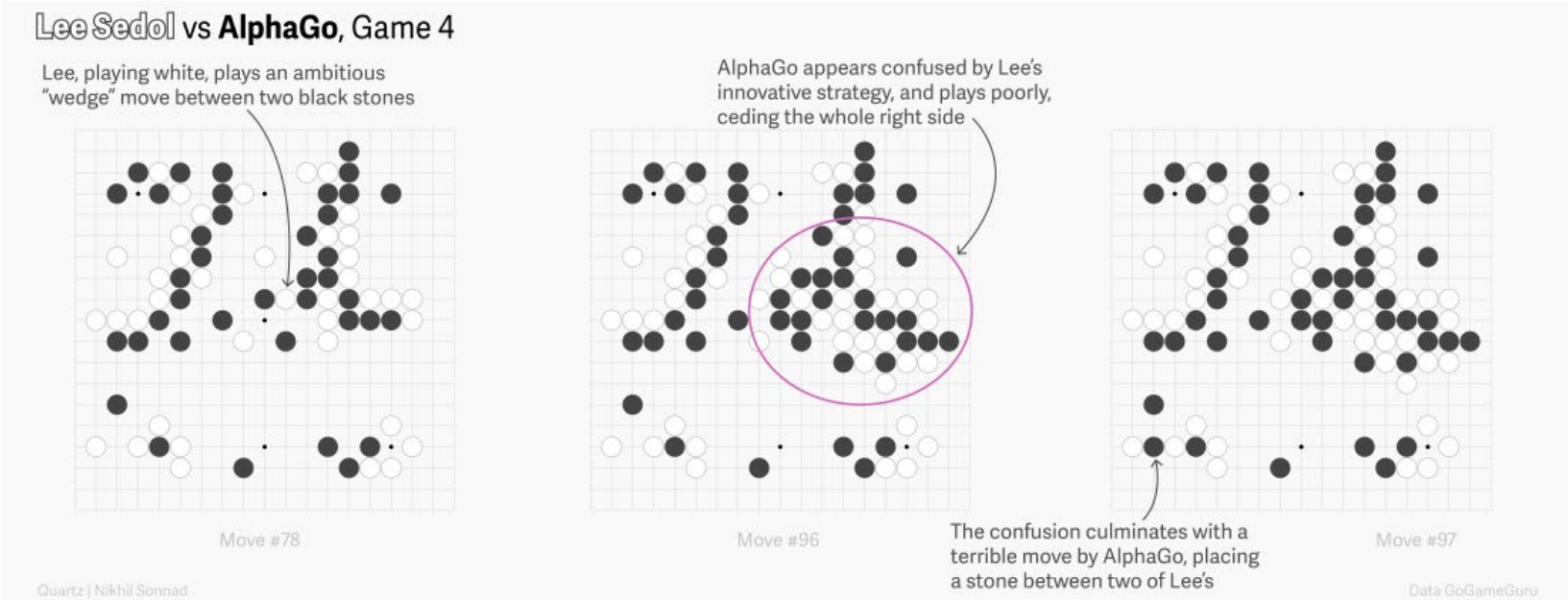
AlphaGo的缔造者，DeepMind的创始人，CEO，Demis Hassabis因为人工智能蛋白质设计工作AlphaFold获得2024诺贝尔化学奖。



2024 年的 ACM A.M. Turing Award（图灵奖）获得者

AlphaGo在围棋游戏中具有超人的能力

AlphaGo(人类水平的感知+超人的决策)>人类?



例外：李世石赢得了与AlphaGo的第4场比赛!

强化学习概述

- 监督学习的上限受到人的策略的限制。
- 强化学习的上限？→ 超越人类策略





AlphaGo *VS* *AlphaZero*

该模型的一部分从人类专家的大型棋谱数据库中进行监督学习。

所有的训练都是自我对弈的强化学习训练，除了游戏的基本规则外，没有任何人类数据或指导

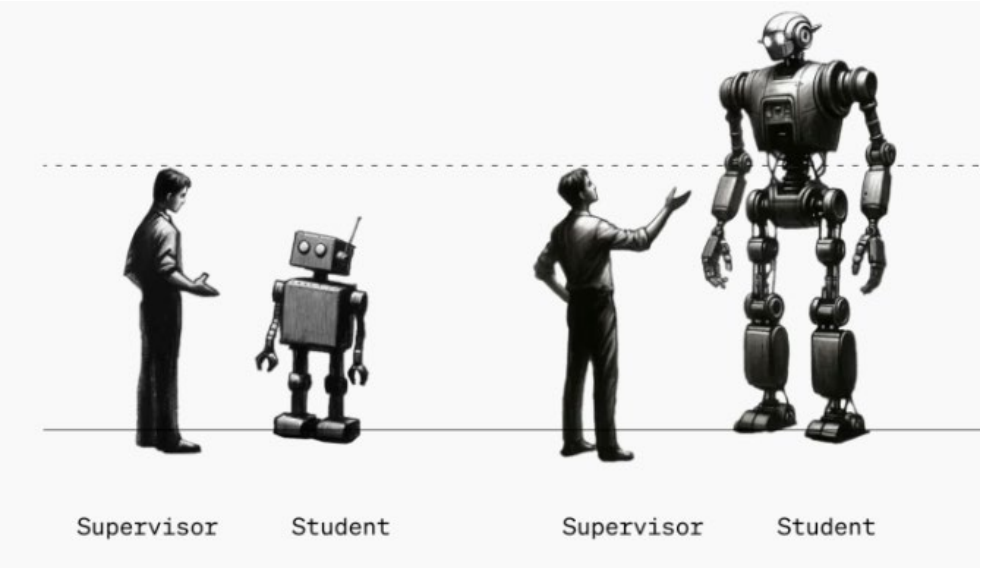
思考

同济大学
TONGJI UNIVERSITY

- 相比监督学习，强化学习为什么可以突破人类策略的上限？

思考

- 相比监督学习，强化学习为什么可以突破人类策略的上限？



当AI的潜力超越人类时，人类的经验无法再为AI提供准确地监督信号，因此，让AI自己从奖励中试错，自己总结出好的policy是一种更好的做法。

强化学习示例——学习走路(游戏)

在游戏中学习走路



强化学习示例——学习竞争

学会在游戏中竞争

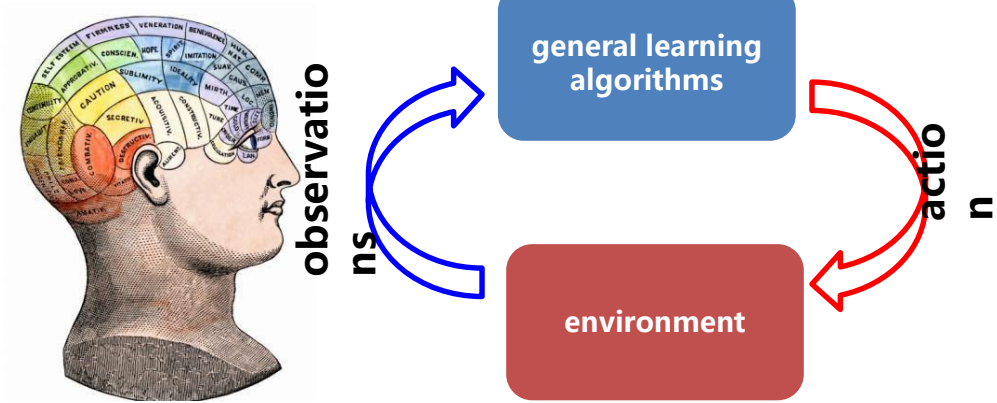


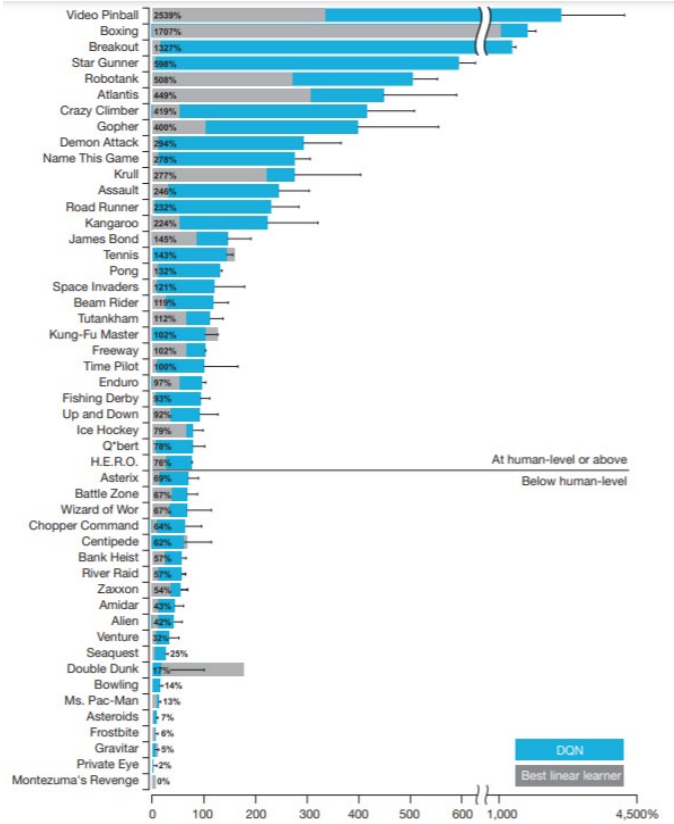
强化学习示例—多智能体学习合作

学会躲猫猫



强化学习是否是AGI的解决方案?





与其尝试制作一个模拟成人思维的程序，为什么不尝试制作一个模拟儿童思维的程序呢?如果经过适当的教育，就会得到成年人的大脑。

——阿兰·图灵

同济大学 TONGJI UNIVERSITY

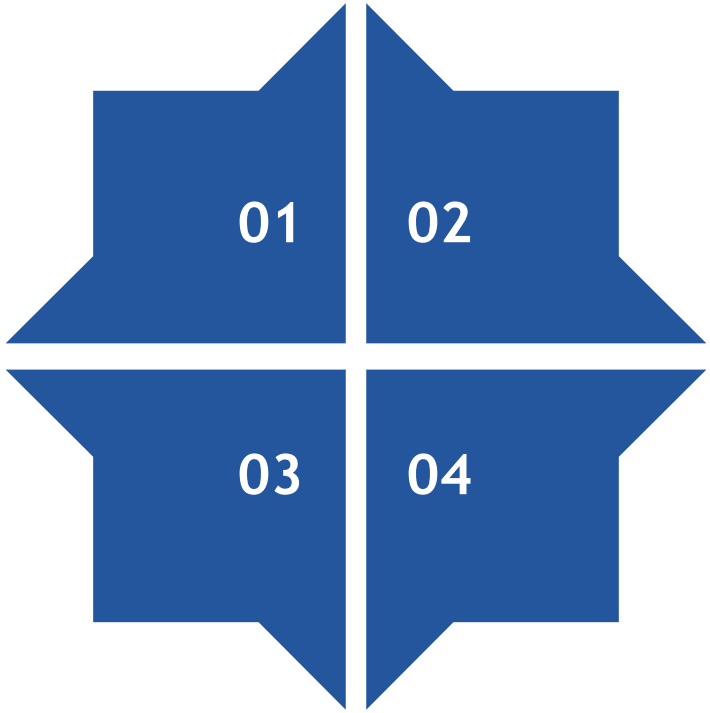
目录

强化学习概述

强化学习的基本概念

强化学习的数学基础

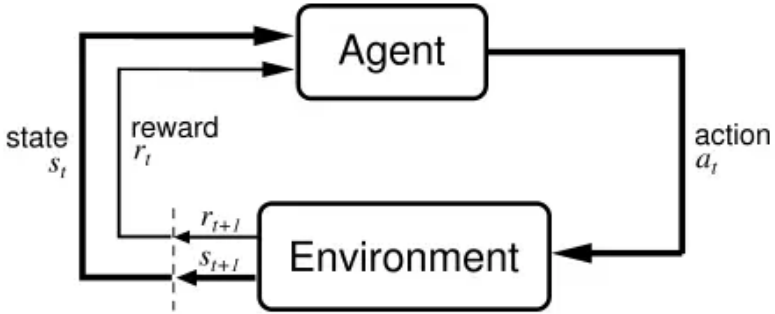
强化学习的定义



强化学习的基本概念

强化学习是智能体Agent通过采取不同的动作与环境Environment之间交互，以奖励达到最大化为目标，学会在不确定环境中进行决策。

强化学习即智能体在环境中不断“试错”，以最大化回报为驱动力，在试错过程中逐渐适应环境，达成学习的目的。



Environment?

强化学习的基本概念

- **强化学习：二部分、三要素是什么?**

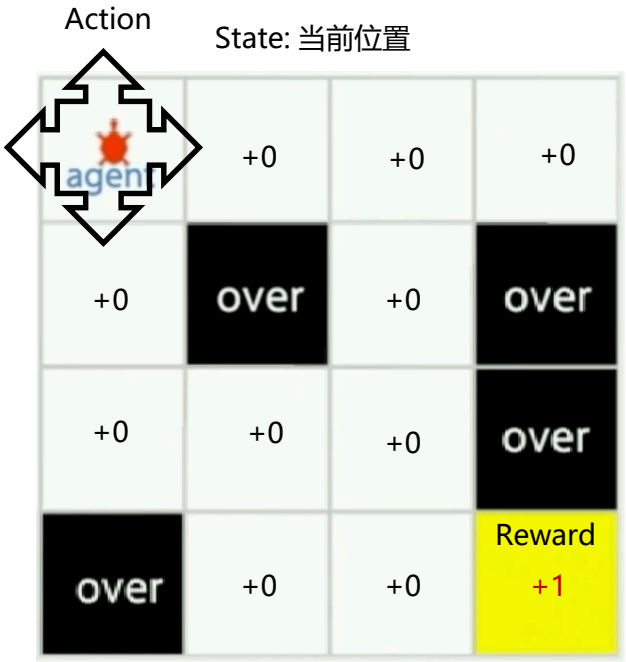
智能体:学习者和决策者

环境:智能体学习并决定执行什么动作的地方

动作:智能体可以执行的一组动作

状态:智能体在环境中的状态

奖励:对于选择的每个行动，环境都会提供奖励

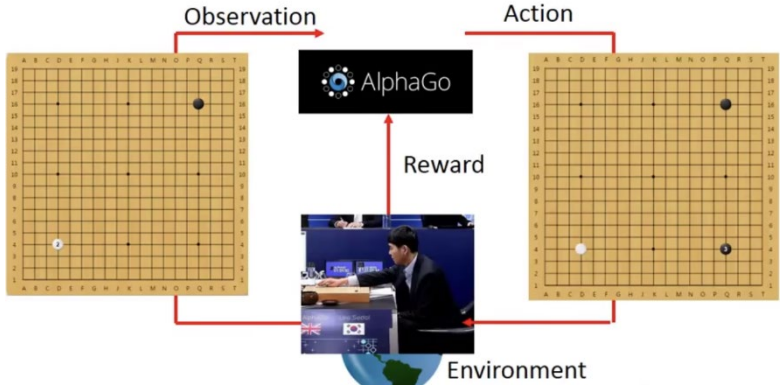


Environment:掉到黑洞游戏结束

强化学习的基本概念

- **智能体（agent）**：智能体是强化学习算法的主体，它能够根据经验做出主观判断并执行动作，是整个智能系统的核心。

- **环境（environment）**：智能体以外的一切统称为环境，环境在与智能体的交互中，能被智能体所采取的动作影响，同时环境也能向智能体反馈状态和奖励。虽说智能体以外的一切都可视为环境，但在设计算法时常常会排除不相关的因素建立一个理想的环境模型来对算法功能进行模拟。



强化学习的基本概念

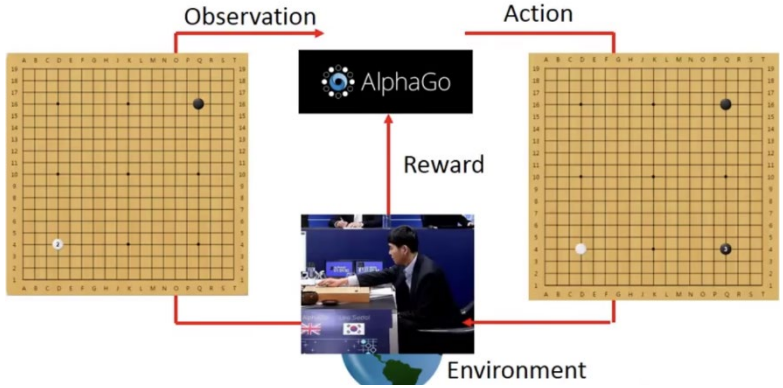
- **环境的类型**很大程度上决定了智能体的设计，以自动驾驶为例：
  - ***部分可观测*** =>Agent不能在每个时间节点上都能获取环境的完整状态
  - ***随机的*** => Agent不得不应对一些突发情况
  - ***多Agent***=>部分合作部分竞争多Agent环境
  - ***动态的*** =>Agent决策时环境会随着时间变化
  - ***连续的***=>连续地执行一系列的控制行动



强化学习的基本概念

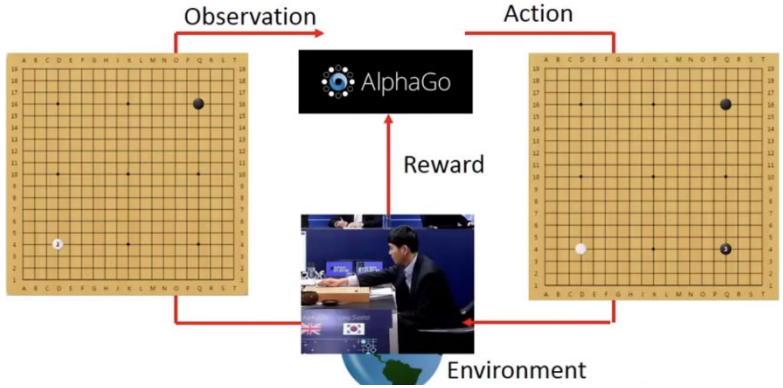
- **状态（state）：** 状态可以理解为智能体对环境的一种理解和编码，通常包含了对智能体所采取决策产生影响的信息。

- **动作（action）：** 动作是智能体对环境产生影响的方式，这里说的动作常常指概念上的动作，如果是在设计机器人时还需考虑动作的执行机构。



强化学习的基本概念

- **奖励（reward）：**奖励是智能体序贯式采取一系列动作后从环境获得的收益。注意奖励概念是现实中奖励和惩罚的统合，一般用正值来代表实际奖励，用负值来代表实际惩罚。



- **奖励信号定义了强化学习问题中的目标**
- **奖励通常是一个标量值，是来自环境反馈的一部分**
- **指出对智能体来说好的和坏的事件是什么**

智能体的唯一目标是在长期运行中最大化它所获得的总奖励。

奖励——举例

- 机器人移动来抓苹果

  获得或丢失苹果的+/-奖励

- 围棋选手下棋就是为了赢

  赢或输的+/-奖励

- 玩雅达利游戏

  增加或减少分数的+/-奖励

- 遇见危险的动物

  逃跑或被吃掉的+/-奖励





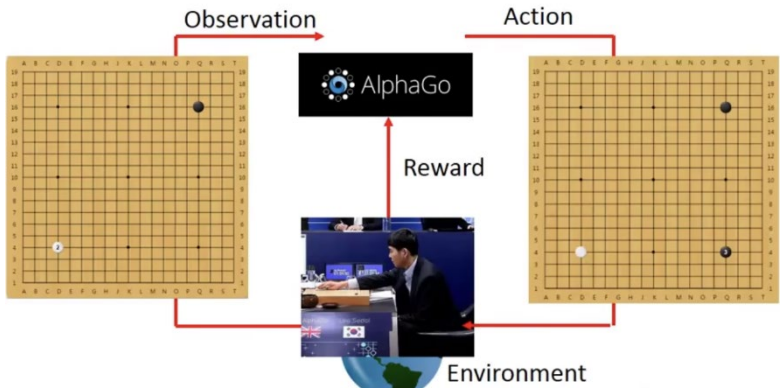
强化学习问题的困难之处在于：在许多情况下，很难定义奖励。

强化学习的基本概念

- **策略（policy）：**策略是智能体在所处状态下执行某个动作的依据，即给定一个状态，智能体可根据一个策略来选择应该采取的动作。

- **模型:智能体对环境的看法，将状态-动作对映射到状态的概率分布**

- **价值函数:从该状态出发并执行特定策略所获得的长期回报**



同济大学 TONGJI UNIVERSITY

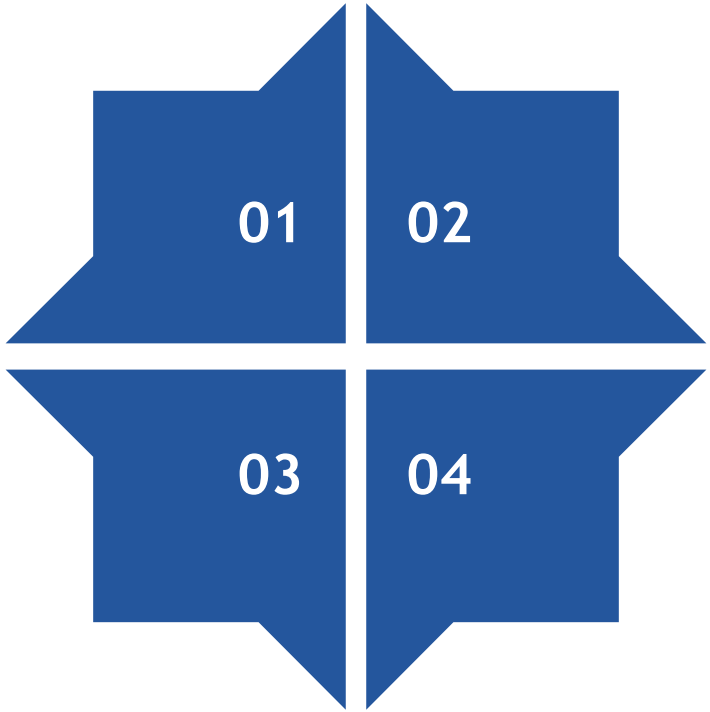
目录

强化学习概述

强化学习的基本概念

强化学习的数学基础

强化学习的定义



马尔可夫决策过程：机器人寻路问题

如图6.2所示3*3的网格，假设有一个机器人位于s_1，试图从s_1这一初始位置向s_9这一目标位置移动。

- 假设机器人每一步只能向上或者向右移动一个方格，到达目标位置s_9则会获得奖励且游戏终止，机器人在移动过程中如果越出方格则会被惩罚且被损坏、并且游戏终止。
- 如何学习一种策略能够帮助机器人从s_1走到s_9？这是一个典型的搜索问题，由此可见强化学习问题与搜索问题存在共通之处。

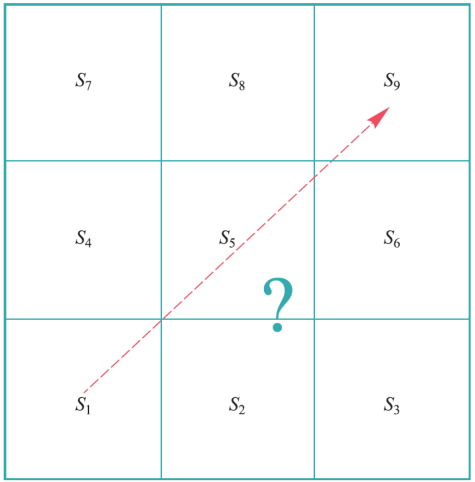


图 6.2　机器人寻路问题

马尔可夫决策过程：机器人寻路问题

智能主体	迷宫机器人
环境	3×3方格
状态	机器人当前时刻所处方格
动作	每次移动一个方格
奖励	到达S_9时给予奖励；越界时给予惩罚

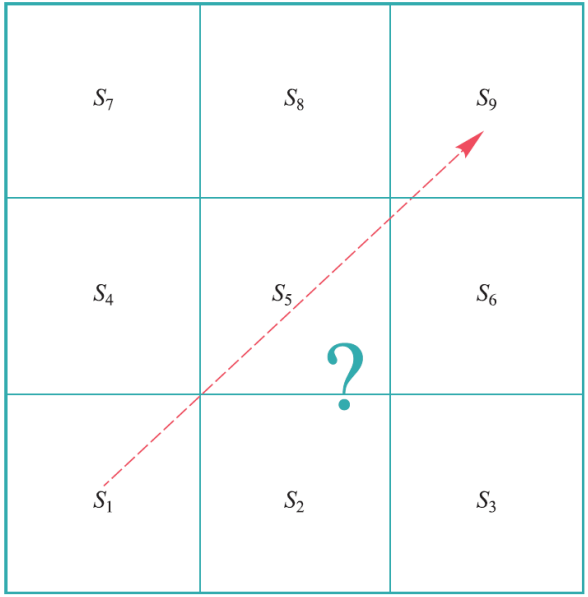


图 6.2　机器人寻路问题

离散马尔可夫链

- 一个随机过程实际上是一列随时间变化的随机变量，其中当时间是离散量时，一个随机过程可以表示为$\{X_t\}_{t=0,1,2,\cdots}$，其中每个X_t都是一个随机变量，这被称为离散随机过程
- 马尔可夫链（Markov Chain）：满足马尔可夫性（Markov Property）的离散随机过程，也被称为离散马尔科夫过程。

$$Pr(X_{t+1} = x_{t+1}|X_0 = x_0, X_1 = x_1, \cdots, X_t = x_t) = Pr(X_{t+1} = x_{t+1}|X_t = x_t)$$

$t+1$时刻状态仅与t时刻状态相关

离散马尔可夫链：引入奖励机制

为了在序列决策中对目标进行优化，在马尔可夫随机过程框架中加入了奖励机制：

- 奖励函数$R: S \times S \mapsto \mathbb{R}$，其中$R(S_t, S_{t+1})$描述了从第$t$步状态转移到第 $t+1$步状态所获得奖励
- 在一个序列决策过程中，不同状态之间的转移产生了一系列的奖励$(R_1, R_2, \cdots)$，其中R_{t+1}为$R(S_t, S_{t+1})$的简便记法。
- 引入奖励机制，这样可以衡量任意序列的优劣，即对序列决策进行评价。

问题：给定两个因为状态转移而产生的奖励序列(1,1,0,0)和(0,0,1,1)，哪个序列决策更好？

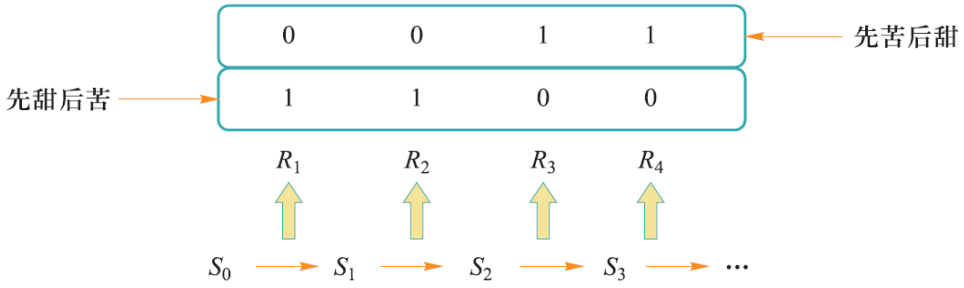


图 6.3　两个奖励序列的比较

离散马尔可夫链：引入回报

为了比较不同奖励机制的优劣，在每个时刻定义**回报（return）**来反映该时刻可得到的累加奖励：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots (6.1.2)$$

其中$\gamma \in [0,1]$是折扣因子（discount factor），$R_{t+k}(t>0)$表示$t+k$时刻获得的奖励。t时刻所获得的回报G_t反映了该时刻之后的累加奖励。当折扣因子γ小于1时，距离当前时刻越远的奖励对该时刻反馈贡献越少。假设$\gamma = 0.99$，则图6.3中两个奖励序列在0时刻的回报值分别为

$$(0,0,1,1)：\ G_0 = 0 + 0.99 \times 0 + 0.99^2 \times 1 + 0.99^3 \times 1 = 1.9504$$

$$(1,1,0,0)：\ G_0 = 1 + 0.99 \times 1 + 0.99^2 \times 0 + 0.99^3 \times 0 = 1.99$$

根据回报值可以认为$(1,1,0,0)$是一个更好的奖励序列。这一结果也与现实生活中的认知活动相符，即距离某个时刻越近，所给予的奖励会对该时刻回报产生更大影响。因此，在强化学习中，要设计的奖励机制应该对当前时刻及其附近时刻能够带来的奖励更为关注。

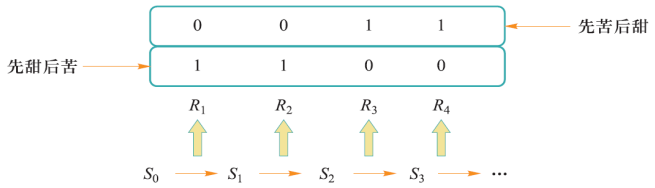


图 6.3　两个奖励序列的比较

马尔可夫奖励过程

在马尔可夫链模型$MP = (S, P)$中加入奖励函数和折扣因子后，可得到的模型被称为马尔可夫奖励过程（Markov reward process，MRP），其形式化定义为$MRP = (S, P, R, \gamma)$。这个模型虽然能够用奖励和回报来刻画智能体的目标，但是仍然不能体现机器人的能动性，即缺乏让机器人与环境进行交互的手段。为了解决这个问题，需要在马尔可夫奖励过程中引入动作，让动作成为机器人影响环境的手段。

马尔可夫决策过程

如下定义马尔可夫决策过程（Markov decision process，MDP）$MDP = (S, A, P, R, \gamma)$

- **状态集合**S：所求解问题中所有可能出现状态所构成的集合，可能是一个有限的集合，也可能是一个无限的集合。
- **动作集合**A：所求解问题中智能体能够采取的所有动作所构成集合，本章所述内容中均假设该集合是一个有限集合。
- **状态转移概率**$P(S_{t+1}|S_t, A_t)$：表示在当前状态S_t下采取了动作A_t后进入下一时刻状态S_{t+1}的概率。显然，状态转移概率满足马尔可夫性。状态转移可以是概率性的（stochastic），也可以是确定的（deterministic）。确定的状态转移指在给定状态S_t下采取了动作A_t后，转移到某一状态的概率为1。
- **奖励函数**$R(S_t, A_t, S_{t+1})$：在状态S_t执行了动作A_t后到达状态S_{t+1}时，智能体能够得到的奖励。
- **折扣因子**γ：后续时刻奖励对当前动作的价值系数，$\gamma \in [0,1]$。

马尔可夫决策过程

如下定义马尔可夫决策过程（Markov decision process，MDP）$MDP = (S, A, P, R, \gamma)$

- 状态集合：$S = \{s_1, s_2, \cdots, s_9, s_d\}$
- 动作集合：$A = \{$上，右$\}$
- 奖励函数：$R(S_t, A_t, S_{t+1}) = \begin{cases} 1, & \text{如果} S_{t+1} = s_9 \\ -1, & \text{如果} S_{t+1} = s_d \\ 0, & \text{其他情况} \end{cases}$
- 折扣因子：$\gamma = 0.99$
- 初始状态$S_0 = s_1$，终止状态集合$S_T = \{s_9, s_d\}$。

$P(S_{t+1}|S_t, a_t=$右$)$

S_t \ S_{t+1}	S_1	S_2	S_3	…	S_9	S_d
S_1	0	1	0	…	0	0
S_2	0	0	1	…	0	0
S_3	0	0	0	…	0	1
…	…			…		…
S_8	0	0	0	…	1	0
S_9	0	0	0	…	0	1

$P(S_{t+1}|S_t, a_t=$上$)$

S_t \ S_{t+1}	S_1	S_4	S_7	…	S_9	S_d
S_1	0	1	0	…	0	0
S_4	0	0	1	…	0	0
S_7	0	0	0	…	0	1
…	…			…		…
S_6	0	0	0	…	1	0
S_9	0	0	0	…	0	1

图 6.4　机器人寻路问题的状态转移函数

−1	−1	−1	
0	0	1	−1
0	0	0	−1
0	0	0	−1

图 6.5　机器人寻路问题的奖惩函数

马尔可夫决策过程：如何与环境交互

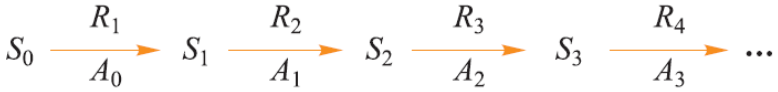


图 6.6　智能体与环境的交互过程

马尔可夫决策过程刻画了智能体与环境的交互特点，这一交互可用图6.6简洁概括。如图所示，智能体从初始状态S_0开始，根据策略执行动作A_0，得到奖励值R_1，同时状态转移到S_1，以此类推。于是，可得到一个状态序列$(S_0, S_1, \cdots,)$，该序列称为**轨迹（trajectory）**，轨迹长度可以是无限的，也可以有终止状态S_T。状态序列中包含终止状态的问题叫**分段（episodic）**问题，不包含终止状态的问题叫**持续（continuing）**问题。在分段问题中，一个从初始状态到终止状态的完整轨迹称为一个**片段（episode）**。

本章讨论的主要是分段问题，有时也将动作和奖励记入状态序列中，用$(S_0, A_0, R_1, S_1, A_2, R_2, S_2, \cdots S_T)$来描述一个片段。

同济大学 TONGJI UNIVERSITY

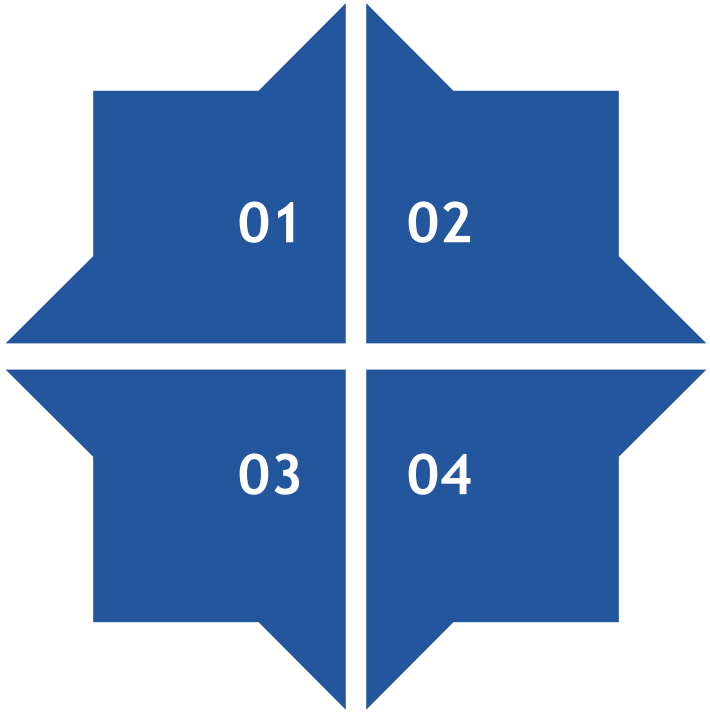
目录

强化学习概述

强化学习的基本概念

强化学习的数学基础

强化学习的定义



强化学习定义

- **策略函数π：**策略函数刻画了智能体选择动作的机制。策略函数$\pi: S \times A \mapsto [0,1]$，$\pi(s,a)$表示智能体在状态$s$下采取动作$a$的概率。策略函数可以是概率性的，也可以是确定的。确定的策略函数指在给定状态s情况下，只有一个动作a使得概率$\pi(s,a)$取值为1。对于确定的策略函数，为了简化符号，记$a = \pi(s)$。一个好的策略函数应该能够使得智能体在采取了一系列行动后可得到最佳奖励，即最大化每一时刻的回报值$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$。从$G_t$的定义可知，$G_t$要根据一次包含了终止状态的轨迹序列计算所得。

- **价值函数（value function）**：$V: S \mapsto \mathbb{R}$，其中$V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$，表示智能体在时刻$t$处于状态$s$时，按照策略$\pi$采取行动时所获得回报的期望。价值函数衡量了某个状态的好坏程度，反映了智能体从当前状态转移到该状态时能够为目标完成带来多大“好处”。

- **动作-价值函数(action-value function)：**$q: S \times A \mapsto \mathbb{R}$，其中$q_\pi(s,a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$，表示智能体在时刻$t$处于状态$s$时，选择了动作$a$后，在$t$时刻后根据策略$\pi$采取行动所获得回报的期望。

强化学习定义

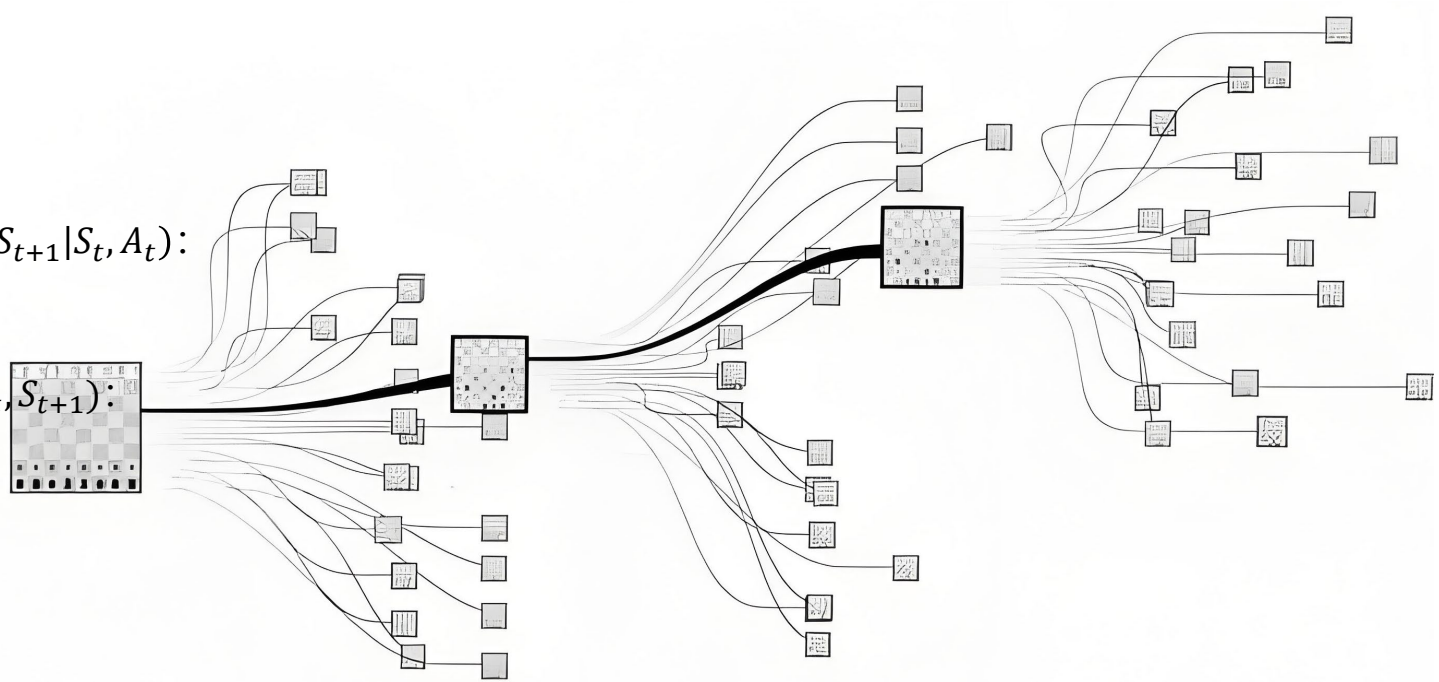
价值函数和动作-价值函数反映了智能体在某一策略下所对应状态序列获得回报的期望，它能比回报本身更加准确地刻画智能体的目标。注意，价值函数和动作-价值函数的定义之所以能够成立，离不开决策过程所具有的马尔可夫性，即当位于当前状态s时，无论当前时刻t的取值是多少，一个策略回报值的期望是一定的（当前状态只与前一状态有关，与时间无关）。

强化学习可以转化为一个策略学习问题，其定义为：给定一个马尔可夫决策过程$MDP = (S, A, P, R, \gamma)$，**学习一个最优策略$\pi^*$，对任意$s \in S$使得$V_{\pi^*}(s)$值最大。**

强化学习的基础

强化学习问题的数学描述：定义马尔可夫决策过程（Markov decision process，MDP）$MDP = (S, A, P, R, \gamma)$

- **状态集合**S：
- **动作集合**A：
- **状态转移概率**$P(S_{t+1}|S_t, A_t)$：
- **奖励函数**$R(S_t, A_t, S_{t+1})$：
- **折扣因子**γ：



强化学习:贝尔曼方程

贝尔曼方程（Bellman Equation）也被称作动态规划方程（Dynamic Programming Equation），由理查德·贝尔曼（Richard Bellman）提出。

- **价值函数（Value Function）** $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s]$
- **动作-价值函数(Action-Value Function)** $q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s, A_t = a]$

$$
\begin{aligned}
V_\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s] \\
&= \mathbb{E}_{a\sim\pi(s,\cdot)}\left[\mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s, A_t = a]\right] \\
&= \sum_{a\in A} \underbrace{\pi(s,a)}_{\text{采取动作}a\text{的概率}} \times \underbrace{q_\pi(s,a)}_{\text{采取动作}a\text{后带来的回报期望}} \\
&= \sum_{a\in A} \pi(s,a) q_\pi(s,a)
\end{aligned}
$$

价值函数取值与时间没有关系，只与策略π、在策略π下从某个状态转移到其后续状态所取得的回报以及在后续所得回报有关。

强化学习:贝尔曼方程

贝尔曼方程（Bellman Equation）也被称作动态规划方程（Dynamic Programming Equation），由理查德·贝尔曼（Richard Bellman）提出。

- **价值函数（Value Function）** $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s]$
- **动作-价值函数(Action-Value Function)** $q_\pi(s,a) =$

$$\mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s, A_t = a]$$

$$q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s, A_t = a]$$

$$= \mathbb{E}_{s' \sim P(\cdot|s,a)}[R(s,a,s')$$

动作-价值函数取值同样与时间没有关系，而是与瞬时奖励和下一步的状态和动作有关。

强化学习:贝尔曼方程

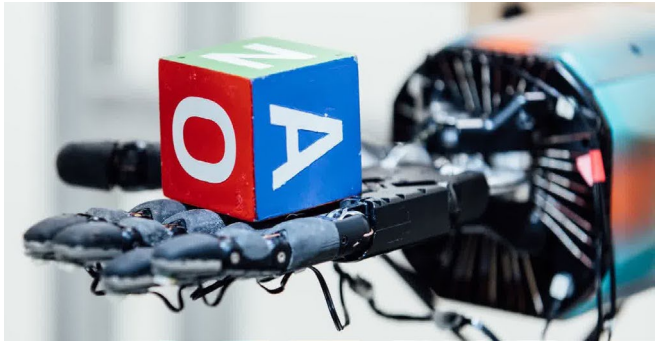
价值函数与动作-价值函数的关系

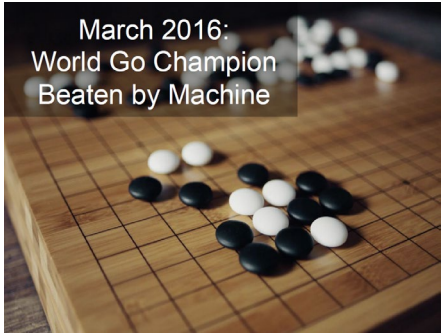
价值函数的贝尔曼方程 $$V_\pi(s) = \mathbb{E}_{a\sim\pi(s,\cdot)}\,\mathbb{E}_{s'\sim P(\cdot|s,a)}[R(s,a,s') + \gamma V_\pi(s')]$$

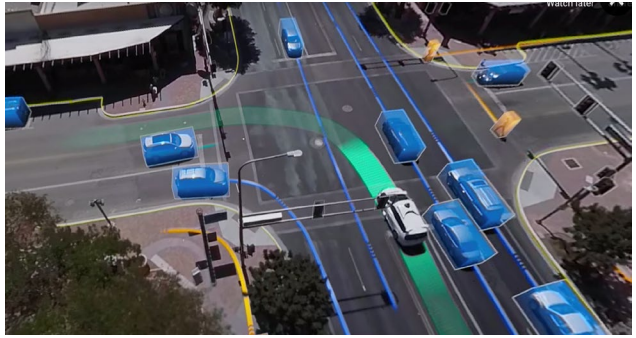
动作-价值函数的贝尔曼方程 $$q_\pi(s,a) = \mathbb{E}_{s'\sim P(\cdot|s,a)}[R(s,a,s') + \gamma \mathbb{E}_{a'\sim\pi(s',\cdot)}[q_\pi(s',a')]]$$

- 贝尔曼方程描述了价值函数或动作-价值函数的递推关系，是研究强化学习问题的重要手段。其中价值函数的贝尔曼方程描述了当前状态价值函数和其后续状态价值函数之间的关系，即当前状态价值函数等于瞬时奖励的期望加上后续状态的（折扣）价值函数的期望。而动作-价值函数的贝尔曼方程描述了当前动作-价值函数和其后续动作-价值函数之间的关系，即当前状态下的动作-价值函数等于瞬时奖励的期望加上后续状态的（折扣）动作-价值函数的期望。
- 在实际中，需要计算得到最优策略以指导智能体在当前状态如何选择一个可获得最大回报的动作。求解最优策略的一种方法就是去求解最优的价值函数或最优的动作-价值函数（即基于价值方法，value-based approach）。一旦找到了最优的价值函数或动作-价值函数，自然而然也就是找到最优策略。当然，在强化学习中还有基于策略（policy-based）和基于模型（model-based）等不同方法。

强化学习
Reinforcement Learning

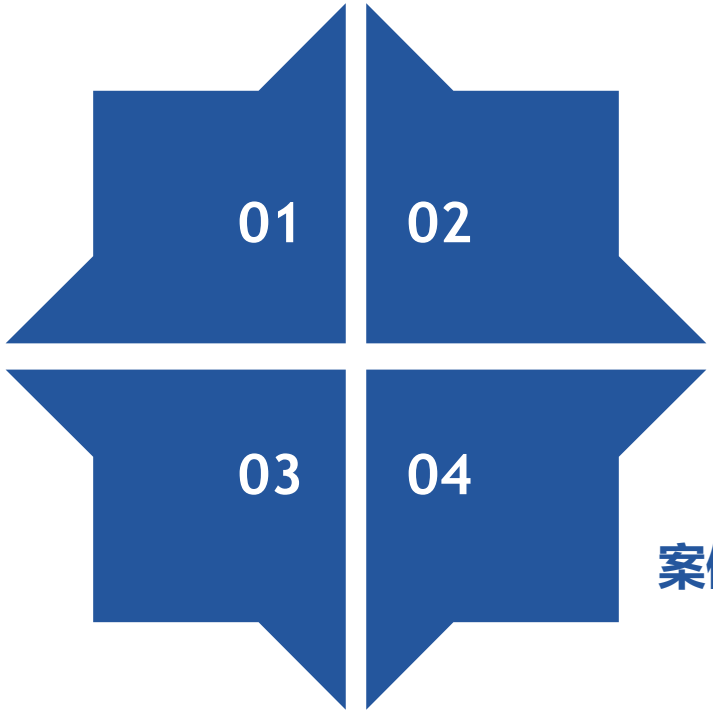






同济大学 TONGJI UNIVERSITY

目录



基于价值的强化学习

基于策略的强化学习

深度强化学习在围棋中的应用

案例分析——AlphaGo

强化学习：基于价值的强化学习

回顾强化学习问题的定义：给定一个马尔可夫决策过程$MDP = (S, A, P, R, \gamma)$，强化学习会寻找一个最优策略$\pi^*$，在策略$\pi^*$作用下使得任意状态$s \in S$对应的价值函数$V_{\pi^*}(s)$取值最大。

为了求解最优策略π^*，图6.7中展示了一种思路：从一个任意的策略开始，首先计算该策略下价值函数（或动作-价值函数），然后根据价值函数调整改进策略使其更优，不断迭代这个过程直到策略收敛。通过策略计算价值函数的过程叫做策略评估（policy evaluation），通过价值函数优化策略的过程叫做策略优化（policy improvement），策略评估和策略优化交替进行的强化学习求解方法叫做通用策略迭代（Generalized Policy Iteration，GPI）。

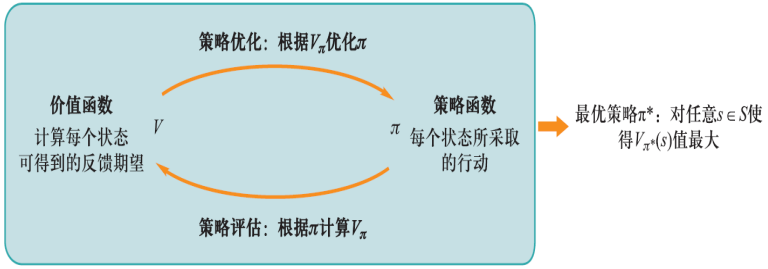


图 6.7　基于通用策略迭代学习最优策略的模式

基于价值的强化学习：策略优化定理

在讨论如何优化策略之前，首先需要明确什么是“更好”的策略。分别给出π和π'两个策略，如果对于任意状态$s \in S$，有$V_\pi(s) \le V_{\pi'}(s)$，那么可以认为策略π'不比策略π差，可见“更优”策略是一个偏序关系。

可以证明，给定任意给定状态$s \in S$，如果两个策略π和π'满足如下条件：

$$q_\pi\big(s,\pi'(s)\big) \ge q_\pi(s,\pi(s))$$

那么对于该任意给定状态$s \in S$，有

$$V_{\pi'}(s) \ge V_\pi(s)$$

即策略π'不比策略π差。这个结论称为策略优化定理（Policy Improvement Theorem）。注意$q_\pi\big(s,\pi'(s)\big)$的含义并不是在当前状态s下按照策略π'行动的回报期望，而是在当前状态s下按照策略π'去选择动作$\pi'(s)$，但是从状态s进入后续状态后，后续状态以后（直至终止状态）的所有动作仍然按照原有策略π来选择所得到的回报期望。注意，按照前面所给出的定义，如果策略π在状态s时只有一个确定的动作可供选择，那么这个动作可以记为$\pi(s)$（$\pi'(s)$也具有同样的含义）。

基于价值的强化学习：策略优化定理

假设当前策略为π，其对应的价值函数和动作-价值函数为V_π和q_π。构造策略π'使得对于任意$s \in S$，有如下条件成立：

$$\pi'(s) = \text{argmax}_a q_\pi(s,a) \quad (6.2.1)$$

于是对于任意$s \in S$，有

$$\begin{aligned} q_\pi\big(s,\pi'(s)\big) &= q_\pi\big(s,\text{argmax}_a\ q_\pi(s,a)\big) \\ &= \max_a q_\pi(s,a) \\ &\geq q_\pi(s,\pi(s)) \end{aligned}$$

根据策略优化定理，可知策略π'不比策略π差，因此策略π'是策略π的一个改进。

基于价值的强化学习：策略优化例子

以图6.8所示机器人寻路问题来介绍策略优化。图中给出了当前策略及其对应价值函数，比如在当前策略下，智能体位于状态s_1的价值函数取值为0.1。当智能体位于s_1状态时，智能体在当前策略指引下将选择向上移动一个方格的行动，从状态s_1进入状态s_4，状态s_4的价值函数取值为0.3。当前策略所给出的其他信息，可从图中周知。

问题：智能体目前在状态s_1，那么下一步是采取“向上移动一个方格”或“向右移动一个方格”哪一个行动?

计算状态s_1选择这两个不同动作后分别所得动作-价值函数取值：

$$q_\pi(s_1,\text{上}) = \sum_{s'\in S} \mathrm{P}(s'|s_1,\text{上})\left[R(s_1,\text{上},s') + \gamma V_\pi(s')\right]$$
$$= 1\times(0+0.99\times 0.3)+0\times\cdots = 0.297$$

$$q_\pi(s_1,\text{右}) = \sum_{s'\in S} \mathrm{P}(s'|s_1,\text{右})\left[R(s_1,\text{右},s') + \gamma V_\pi(s')\right]$$
$$= 1\times(0+0.99\times 0.4)+0\times\cdots = 0.396$$

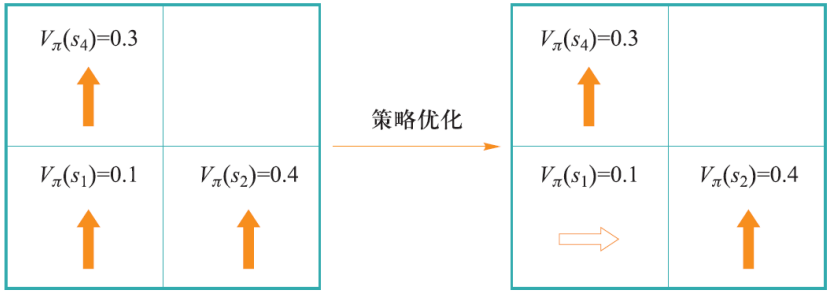


图 6.8　对状态 s_1 所采取的行动进行策略优化的示意

基于价值的强化学习：策略优化例子

问题：智能体目前在状态s_1，那么下一步是采取“向上移动一个方格”或“向右移动一个方格”哪一个行动?

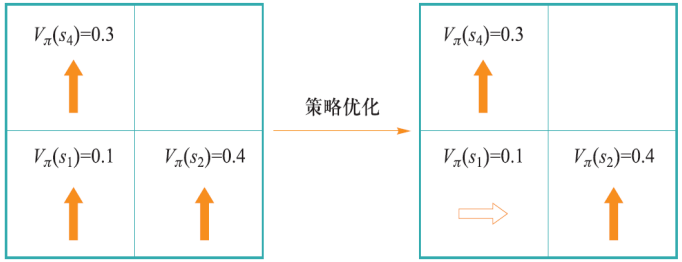


图 6.8　对状态 s_1 所采取的行动进行策略优化的示意

应该选择“向右移动一个方格”行动

- 智能体在状态s_1选择“向上移动一个方格”行动所得回报$q_\pi(s_1,$上$)$值为0.297、选择“向右移动一个方格”行动所得回报$q_\pi(s_1,$右$)$值为0.396。
- 显然，智能体在状态s_1应该选择“向右移动一个方格”行动，这样能够获得更大的回报。
- 经过策略优化后，状态s_1处的新策略为$\pi'(s_1)=\text{argmax}_a q_\pi(s,a)=$右，则将$s_1$处的策略从“上”更新为 “右”。其他状态的情况可用类似方法计算得到。

基于价值的强化学习：策略评估

假定当前策略为π，策略评估指的是根据策略π来计算相应的价值函数V_π或动作-价值函数q_π。

- 动态规划
- 蒙特卡洛采样
- 时序差分（Temporal Difference）

介绍在状态集合有限前提下三种常见的策略评估方法，它们分别是基于动态规划的方法、基于蒙特卡洛采样的方法和时序差分（temporal difference）法。

策略评估：动态规划

基于动态规划的价值函数更新：使用迭代的方法求解贝尔曼方程组

初始化V_π函数
循环
　　枚举$s \in S$
　　$V_\pi(s) \leftarrow \sum_{a\in A}\pi(s,a)\sum_{s'\in S}Pr(s'|s,a)\,[R(s,a,s')+\gamma V_\pi(s')]$

$q_\pi(s,a)$ 等同于

直到V_π收敛

更新$V_\pi(s_1)$的值：

$$q_\pi(s_1,\text{上}) = 1\times(0+0.99\times 0.3)+0\times(0+0.99\times 0.4)$$

$$+\ldots = 0.297$$

$$V_\pi(s_1) = 1\times q_\pi(s_1,\text{上})+0\times q_\pi(s_1,\text{右}) = 0.297$$

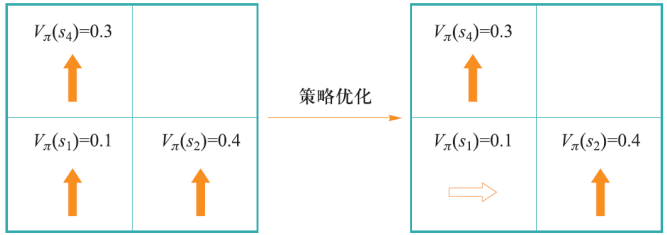


图 6.8　对状态 s_1 所采取的行动进行策略优化的示意

动态规划法的缺点：智能主体需要事先知道状态转移概率；无法处理状态集合大小无限的情况

策略评估：蒙特卡洛采样

基于蒙特卡洛采样的价值函数更新

选择不同的起始状态，按照当前策略π采样若干轨迹，记它们的集合为D

枚举$s \in S$

计算D中s每次出现时对应的反馈$G_1, G_2, \cdots, G_k$

$V_\pi(s) \leftarrow \frac{1}{k}\sum_{i=1}^{k} G_i$

假设按照当前策略可样得到以下两条轨迹

$(s_1, s_4, s_7, s_8, s_9)$

(s_1, s_2, s_3, s_d)

s_1对应的反馈值分别为

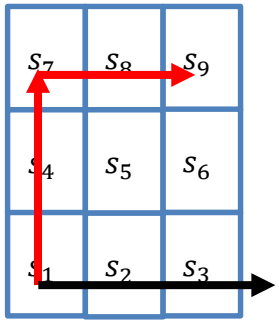
$$0 + \gamma \times 0 + \cdots + \gamma^3 \times 1 = 0.970$$

$$0 + \gamma \times 0 + \gamma^2 \times (-1) = -0.980$$

因此估计

$$V(s_1) = \frac{1}{2}(0.970 - 0.980) = -0.005$$

如果是确定的策略，每个起点只会产生一种轨迹



根据数理统计的知识，期望可以通过样本均值来估计的，这正是蒙特卡洛方法的核心思想。即大数定理指出：对于独立同分布（的样本数据，当样本足够大的时候，样本平均值向期望值收敛。按照这样的思路可使用蒙特卡洛方法来进行策略评估：给定状态s，从该状态出发不断采样后续状态，得到不同的采样序列。通过这些采样序列来分别计算状态s的回报值，对这些回报值取均值，作为对状态s价值函数的估计，从而避免对状态转移概率的依赖。

策略评估：时序差分

基于时序差分（Temporal Difference）的价值函数更新

初始化V_π函数
循环
　　初始化s为初始状态
　　循环
　　　　$a \sim \pi(s,\cdot)$
　　　　执行动作a，观察奖励R和下一个状态s'
　　　　更新$V_\pi(s) \leftarrow V_\pi(s) + \alpha[R(s,a,s') + \gamma V_\pi(s') - V_\pi(s)]$
　　　　$s \leftarrow s'$
　　直到s是终止状态
直到V_π收敛

时序差分法可以看作蒙特卡罗方法和动态规划方法的有机结合。时序差分算法与蒙特卡洛方法相似之处在于，时序差分方法从实际经验中获取信息，无需提前获知环境模型的全部信息。时序差分算法与动态规划方法的相似之处在于，时序差分方法能够利用前序已知信息来进行在线实时学习，无需等到整个片段结束（终止状态抵达）再进行价值函数的更新。

策略评估：时序差分

算法 6.3 基于时序差分法的策略评估

函数： TemporalDifference
输入： 策略 π
输出： 价值函数 V_π

随机初始化 V_π
repeat
  $s \leftarrow$初始状态
  repeat
    $a \sim \pi(s, \cdot)$
    执行动作 a，观察奖励 R 和下一个状态 s'
    $V_\pi(s) \leftarrow V_\pi(s) + \alpha[R + \gamma V_\pi(s') - V_\pi(s)]$
    $s \leftarrow s'$
  until s 是终止状态
until V_π 收敛

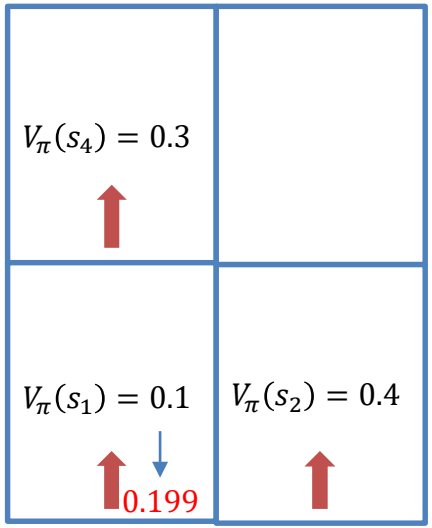
更新$V_\pi(s)$的值： $V_\pi(s) \leftarrow (1-\alpha)V_\pi(s) + \alpha[R(s,a,s') + \gamma V_\pi(s')]$

过去的价值函数值　　**学习得到的价值函数值**

- 动态规划法根据贝尔曼方程迭代更新价值函数，要求算法事先知道状态之间的转移概率，这往往是不现实的。为了解决这个问题，时序差分法借鉴蒙特卡洛法思想，通过采样a和s'来估计计算$V_\pi(s)$
- 由于通过采样进行计算，所得结果可能不准确，因此时序差分法并没有将这个估计值照单全收，而是以α作为权重来接受新的估计值，即把价值函数更新为$(1-\alpha)V_\pi(s) + \alpha[R + \gamma V_\pi(s')]$，对这个式子稍加整理就能得到算法6. 3中第7行形式：$V_\pi(s) \leftarrow V_\pi(s) + \alpha[R + \gamma V_\pi(s') - V_\pi(s)]$。这里$R + \gamma V_\pi(s')$为时序差分目标，$R + \gamma V_\pi(s') - V_\pi(s)$为时序差分偏差。
- 时序差分法和蒙特卡洛法都是通过采样若干个片段来进行价值函数更新的，但时序差分法并非使用一个片段中的终止状态所提供的实际回报值来估计价值函数，而是根据下一个状态的价值函数来估计，这样克服了采样轨迹的稀疏性可能带来样本方差较大的不足问题，同时缩短了反馈周期。

策略评估：时序差分

初始化V_π函数
循环
　　初始化s为初始状态
　　循环
　　　　$a \sim \pi(s,\cdot)$
　　　　执行动作a，观察奖励R和下一个状态s'
　　　　更新$V_\pi(s) \leftarrow V_\pi(s) + \alpha[R(s,a,s') + \gamma V_\pi(s') - V_\pi(s)]$
　　　　$s \leftarrow s'$
　　直到s是终止状态
直到V_π收敛



在对片段进行采样的同时，不断以上述方法更新当前状态的价值函数，不断迭代直到价值函数收敛为止。

假设$\alpha = 0.5$，更新$V_\pi(s_1)$的值：
从$\pi(s_1,\cdot)$ 中采样得到动作$a =$ 上
从$Pr(\cdot|s_1,$上$)$中采样得到下一步状态$s' = s_4$

$$V_\pi(s_1) \leftarrow V_\pi(s_1) + \alpha[R(s_1,\text{上},s_4) + \gamma V_\pi(s_4) - V_\pi(s_1)]$$
$$= 0.1 + 0.5 \times [0 + 0.99 \times 0.3 - 0.1] = 0.199$$

策略评估：基于价值的强化学习算法

算法 6.4 直观构造的强化学习算法

函数： PolicyIteration

输入： 马尔可夫决策过程 MDP=(S, A, P, R, γ)

输出： 策略 π

随机初始化 π

repeat

  $q_\pi \leftarrow \text{PolicyEvaluation}(\pi)$

  foreach $s \in S$ **do**

    $\pi(s) \leftarrow \text{argmax}_a q_\pi(s, a)$

  end

until π 收敛

算法 6.5 价值迭代算法

函数： ValueIteration

输入： 马尔可夫决策过程 MDP=(S, A, P, R, γ)

输出： 策略 π

随机初始化 V_π

repeat

  foreach $s \in S$ **do**

    $V_\pi(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a)[R(s, a, s') + \gamma V_\pi(s')]$

  end

until V_π 收敛

$\pi(s)$: $\text{argmax}_a \sum_{s'} P(s' \mid s, a)[R(s, a, s') + \gamma V_\pi(s')]$

在策略评估动态规划法的基础上，每次迭代只对一个状态进行策略评估和策略优化，就可以得到算法6.5，这个算法被称为价值迭代（value iteration）算法

策略评估：基于价值的强化学习算法

算法 6.5 价值迭代算法

函数： ValueIteration

输入： 马尔可夫决策过程 MDP=(S, A, P, R, γ)

输出： 策略 π

1 随机初始化 V_π

2 **repeat**

3 | **foreach** $s \in S$ **do**

4 | | $V_\pi(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)[R(s,a,s')+\gamma V_\pi(s')]$

5 | **end**

6 **until** V_π 收敛

7 $\pi(s) := \arg\max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_\pi(s')]$

算法6.5中的第4行可以拆分成如下两步，即：

$$\pi(s) \leftarrow \text{argmax}_a q_\pi(s,a)$$

$$V_\pi(s) \leftarrow q_\pi(s,\pi(s))$$

其中$q_\pi(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_\pi(s')]$。这两步分别对应了策略优化和策略评估两个阶段。也就是说，在算法6.5的内层循环中，就已经交替进行了一次策略优化和一次策略评估。

策略评估：基于价值的强化学习算法

以机器人寻路问题为例，假设价值函数的初值如图6.9(a)所示，则完成算法6.5中一次外层循环后，价值函数的值如图6.9(b)所示。

以状态s_8为例：

$$q_\pi(s_8,\text{右}) = \sum_{s'} P(s'|s_8,\text{右})[R(s_8,\text{右},s') + \gamma V_\pi(s')]$$
$$= 1 \times (1 + 0.99 \times 0) + 0 \times \cdots = 1$$

$$q_\pi(s_8,\text{上}) = \sum_{s'} P(s'|s_8,\text{上})[R(s_8,\text{上},s') + \gamma V_\pi(s')]$$
$$= 1 \times (-1 + 0.99 \times 0) + 0 \times \cdots = -1$$
$$V(s_8) = \max_a q_\pi(s_8,a) = \max\{1,-1\} = 1$$

因此状态s_8的价值函数更新为1，同理可更新其他状态的价值函数。图6.9(c)和图6.9(d)分别为完成了两次和三次外层循环后的价值函数，而图6.9(e)则为完成5次外层循环后的结果，这个结果已经收敛。

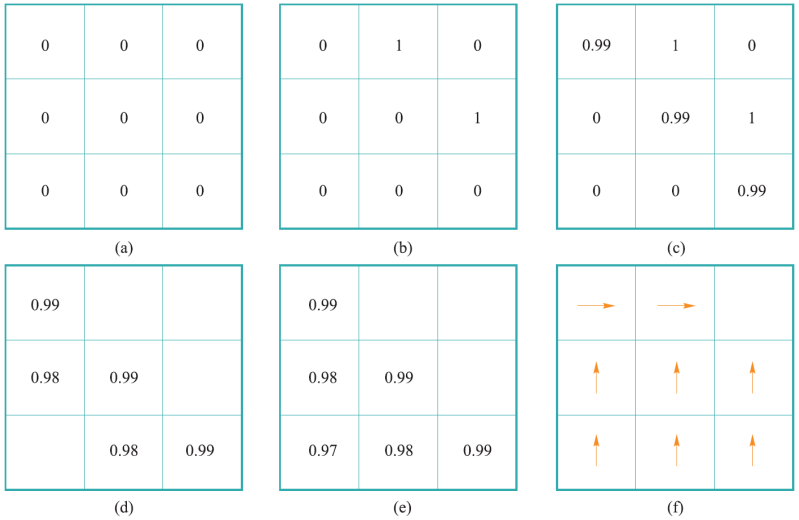


图 6.9　价值迭代法的求解过程

由于s_8向右移动一个方格进入终止状态会获得1这样的回报，在价值迭代过程中，可以看到这会使得产生这一回报的状态由于选择了相应行动而获得相应价值，如状态s_8的价值函数从初始时取值0更新为1（见图 6.9(a)和图6.9(b)）。图6.9(f)给出了根据图6.9(e)计算出的策略（动作-价值相同的情况优先取动作“上”）。可见智能体从s_1出发，根据图6.9(f)中的策略，可以沿着轨迹$(s_1, s_4, s_7, s_8, s_9)$到达目标。

策略评估：Q学习算法

初始化q_π函数
循环
　　初始化s为初始状态
　　循环
　　~~$a \sim \pi(s,)$~~　　$a = \text{argmax}_{a'} q_\pi(s,a)$
　　执行动作a，观察奖励R和下一个状态s'
　　~~更新$V_\pi(s) \leftarrow V_\pi(s) + \alpha[R + \gamma V_\pi(s') - V_\pi(s)]$~~
　　更新$q_\pi(s,a) \leftarrow q_\pi(s,a) + \alpha\left[R + \gamma \max_{a'} q_\pi(s',a') - q_\pi(s,a)\right]$
　　$s \leftarrow s'$
　　直到s是终止状态
直到q_π收敛

先前的策略优化：$\pi'(s) = \text{argmax}_a q_\pi(s,a)$

$$q_\pi(s,a) \leftarrow (1-\alpha) q_\pi(s,a) + \alpha\left[R + \gamma \max_{a'} q_\pi(s',a')\right]$$

算法 6.6　Q 学习算法

函数： QLearning
输入： 马尔可夫决策过程 MDP=(S, A, P, R, γ)
输出： 策略 π

```
随机初始化 q_π
repeat
    s←初始状态
    repeat
        a←argmax_a' q_π( s, a' )
        执行动作 a，观察奖励 R 和下一个状态 s'
        q_π( s, a )←q_π( s, a )+α[ R+γmax_a' q_π( s', a' )−q_π( s, a ) ]
        s←s'
    until s 是终止状态
until q_π 收敛
π( s ) : argmax_a q( s, a )
```

Q学习中直接记录和更新动作-价值函数q_π而不是价值函数V_π，这是因为策略优化要求已知动作-价值函数q_π，如果算法仍然记录价值函数V_π，在不知道状态转移概率的情况下将无法求出q_π。于是，Q学习中，只有动作-价值函数（即q函数）参与计算。

策略评估：Q学习算法

- 图6.10(a)表示算法的初始状态，其中a/b表示对应状态的动作-价值函数的取值，斜线上方的a表示$q_\pi(s,\text{上})$，斜线右侧的b表示$q_\pi(s,\text{右})$。
- 图6.10(a)中显示除终止状态外的所有的向上的动作-价值函数值为0.2，所有向右的动作-价值函数值为0。当然，令这些动作-价值函数的初始值全部为0也是可以的，图6.10(a)中所设置的初始值只是为了让这个例子更容易说明策略学习。

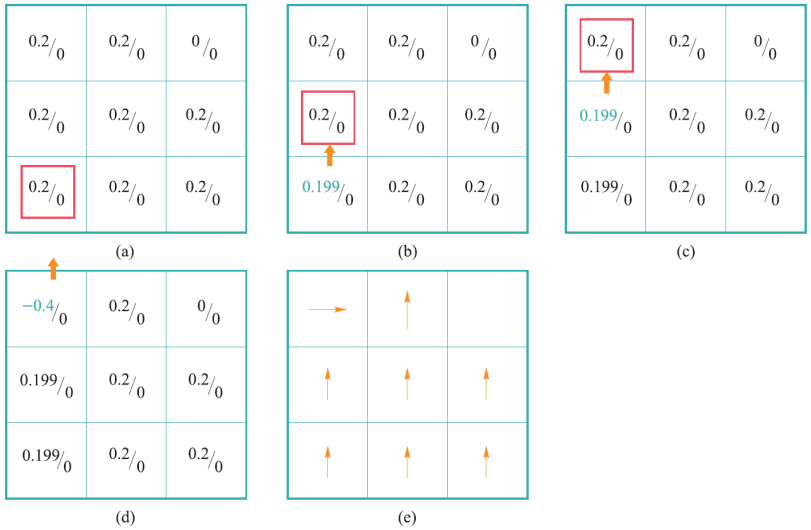


图 6.10　Q 学习的一个片段的执行过程

策略评估：Q学习算法

图6.10(a)中的红色方框代表了智能体当前所处的状态位置。从图6.10(a)为出发，在算法第5行，根据当前策略求出智能体应该采取的动作$a = \text{argmax}_a q_\pi(s_1, a) =$ 上，执行这个动作，得到奖励$R = 0$和进入下一状态$s' = s_4$，因此可如下更新对应的动作-价值函数：

$$q_\pi(s_1, \text{上}) \leftarrow q_\pi(s_1, \text{上}) + \alpha[R + \gamma \max_{a'} q_\pi(s', a') - q_\pi(s, a)]$$
$$= 0.2 + 0.5 \times [0 + 0.99 \times \max\{0, 0.2\} - 0.2] = 0.199$$

同时令当前状态为s_4，由此得到了图6.10(b)。同理算法再执行一次内层循环得到图6.10(c)，接着得到图6.10(d)。这个过程中每一步都通过后续状态的动作-价值函数更新当前状态的动作-价值函数，图6.10(b)-(c)都没有对策略产生影响。

当智能体在图6.10(d)中到达了损坏状态s_d，一个负的奖励（即奖励为-1）使$q_\pi(s_7, \text{上})$变为负值，此时策略$\pi(s_7)$从“向上移动一个方格”变成了“向右移动一个方格”，至此智能体完成了一个片段（一次外层循环）的更新。

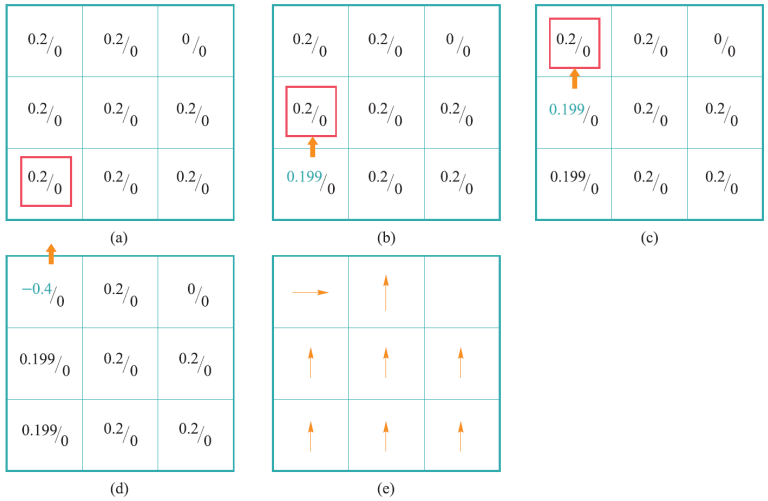


图 6.10　Q 学习的一个片段的执行过程

策略评估：Q学习算法

图6.11展示了Q学习算法执行三个片段的过程，从第二个片段结束时起，算法已经学习得到了沿着轨迹$(s_1, s_4, s_7, s_8, s_9)$到达目标状态$s_9$的策略。实际上，虽然此时动作-价值函数尚未收敛，但此后策略已经不会再发生变化了。

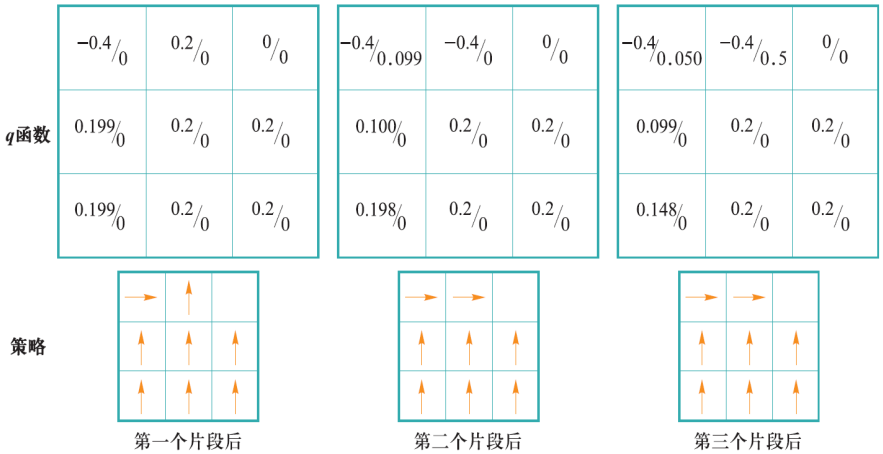


图 6.11　Q 学习执行三个片段的过程

策略评估：探索与利用

如果将动作-价值函数初始化为$1/-2$，即除了终止状态，对于每个状态，智能体更倾向于向上移动一个方格而非向右一个方格。在这样初始策略下，Q学习执行过程如图6.12所示，可见即使在Q学习算法收敛以后，所得到的策略仍然不能使得机器人抵达目标状态s_9，而是沿着轨迹(s_1, s_4, s_7, s_d)导致机器人被损坏。

图6.12中问题出现的原因概括来说就是$q_\pi(\cdot, 右)$的初始值太小，导致机器人被损坏产生的-1奖励不足以推动智能体来改变策略。既然外部刺激不足以使机器人尝试新的策略，那么不妨从内部入手为智能体改变固有策略来添加一个探索的动力。

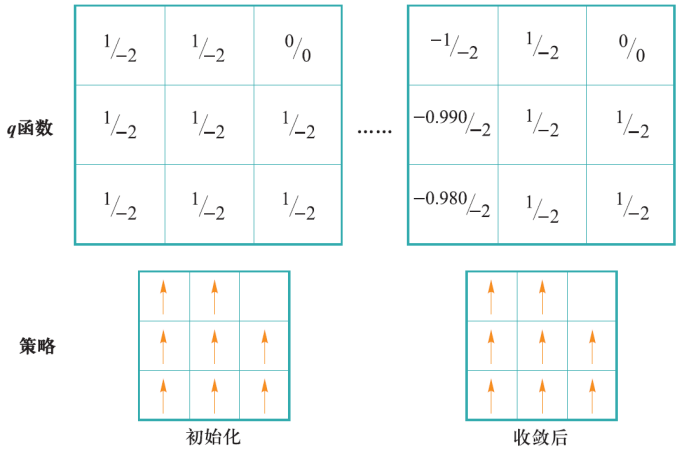


图 6.12　动作 – 价值函数初始化为 1/−2 时 Q 学习算法的执行过程

策略评估：探索与利用

初始化q_π函数
循环
　　初始化s
　　循环

ϵ贪心（ϵ-greedy）策略

$$\epsilon-greedy_\pi(s) = \begin{cases} \text{argmax}_a q_\pi(s,a), & \text{以}1-\epsilon\text{的概率} \\ \text{随机的} a \in A, & \text{以}\epsilon\text{的概率} \end{cases}$$

$a = \text{argmax}_{a'} q_\pi(s,a')$

用ϵ贪心（ϵ-greedy）策略
代替$a = \text{argmax}_{a'} q_\pi(s,a')$

执行动作a，观察奖励R和下一个状态s'

更新$q_\pi(s,a) \leftarrow q_\pi(s,a) + \alpha\left[R + \gamma \max_{a'} q_\pi(s',a') - q_\pi(s,a)\right]$

$s \leftarrow s'$

　　直到s是终止状态
直到q_π收敛

在Q学习中引入探索（exploration）与利用（exploitation）机制。这一机制用用ϵ贪心（ϵ-greedy）策略来代替$a = \text{argmax}_{a'} q_\pi(s,a')$。用$\epsilon$贪心（$\epsilon$-greedy）策略定义如下：在状态s,以$1-\epsilon$的概率来选择带来最大回报的动作，或者以$\epsilon$的概率来随机选择一个动作。

策略评估：探索与利用

算法 6.7 使用 ϵ 贪心算法进行探索的 Q 学习

函数： EpsGreedy

输入： 状态 s，动作－价值函数 q_π，参数 ϵ

输出： 动作 a

$n \sim \text{uniform}(0, 1)$
if $n < \epsilon$ **then**
  $a \leftarrow$从 A 中随机选择
else
  $a \leftarrow \arg\max_{a'} q_\pi(s, a')$
end

函数： QLearning

输入： 马尔可夫决策过程 $\text{MDP}=(S, A, P, R, \gamma)$

输出： 策略 π

随机初始化 q_π
repeat
  $s \leftarrow$初始状态
  repeat
    $a \leftarrow \text{EesGreedy}(s, q_\pi, \epsilon)$
    执行动作 a，观察奖励 R 和下一个状态 s'
    $q_\pi(s, a) \leftarrow q_\pi(s, a) + \alpha[R + \gamma \max_{a'} q_\pi(s', a') - q_\pi(s, a)]$
    $s \leftarrow s'$
  until s 是终止状态
until q_π 收敛
$\pi(s) := \arg\max_a q(s, a)$

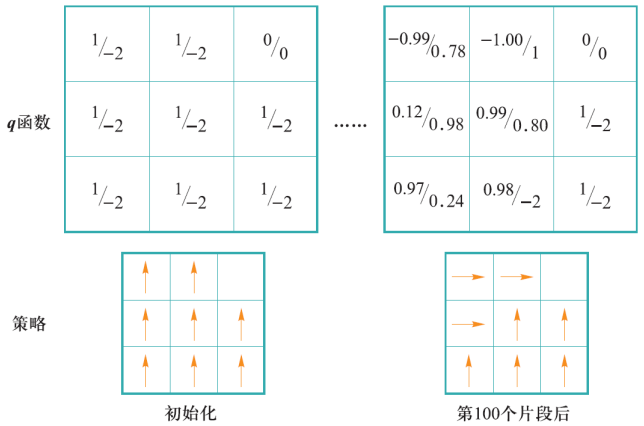


图 6.13 使用 ϵ 贪心策略进行探索的 Q 学习的执行过程

从图6.13可知，算法在执行了100个片段后，得到了经过轨迹$(s_1, s_4, s_5, s_8, s_9)$到达目标状态的策略。由于$\epsilon$贪心策略具有概率性，因此图6.13中的结果并不是确定的，有可能经过100个片段仍不能得到一个合适的策略，且在图6.13中也能发现存在还没有被访问过的状态s_3。但随着迭代次数的增加，算法探索到最优策略的可能性也会显著增加，通过适当增大参数ϵ的值也能够促进算法乐于探索。

策略评估： 参数化与深度强化学习

- 考虑在算法6.7中给出的Q学习算法流程。在这个算法中，动作-价值函数需要用一个数组来记录。但是在有些实际问题中，状态数量会非常多甚至是无限的，这样会造成如下两个问题：首先是算法很难用一个数组来存储动作-价值函数的值；其次是有些状态的访问次数可能很少甚至根本没有被访问过，这些状态的价值估计是不可靠的。
- 为了解决这两个问题，一种解决方案是将动作-价值函数参数化（parametrized），即用一个回归模型来拟合q_π函数。如果回归模型是一个深度神经网络，那么这样的算法就被称为深度强化学习(deep reinforcement learning)算法。

算法 6.8 参数化的 Q 学习算法

函数： DeepQLearning

输入： 马尔可夫决策过程 $\mathrm{MDP}=(S, A, P, R, \gamma)$

输出： 策略 π

随机初始化 q_π 的参数 θ

repeat

  $s \leftarrow$初始状态

  repeat

    $a \leftarrow \mathrm{EpsGreedy}(s, \boldsymbol{\theta}, \epsilon)$

    执行动作 a，观察奖励 R 和下一个状态 s'

    $L(\theta) \leftarrow -\frac{1}{2}[R+\gamma \max_a q_\pi(s', a'; \boldsymbol{\theta}) - q_{\pi\pi}(s, a; \theta)]^2$

    $\theta \leftarrow \theta - \eta \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$

    $s \leftarrow s'$

  until s 是终止状态

until q_π 收敛

$\pi(s) := \arg\max_a q(s, a)$

策略评估： 参数化与深度强化学习

- 和非参数化的算法6.7进行对比，不难发现，区别主要在算法6.8中第7行和第8行的动作-价值函数更新。
- 类似6.2.3节对时序差分法的分析，$R+\max_{a'} q_\pi(s',a';\boldsymbol{\theta})$是对$q_\pi(s,a)$的估计值，因此算法6. 8中第7行计算的是$q_\pi(s,a)$的估计值和当前值之间误差的平方，第8行则是将该值作为损失函数，利用梯度下降法来更新参数$\boldsymbol{\theta}$。

算法 6.8 参数化的 Q 学习算法

函数： DeepQLearning

输入： 马尔可夫决策过程 MDP $=(S, A, P, R, \gamma)$

输出： 策略 π

1 随机初始化 q_π 的参数 θ

2 **repeat**

3 　$s \leftarrow$初始状态

4 　**repeat**

5 　　$a \leftarrow \text{EpsGreedy}(s, \boldsymbol{\theta}, \epsilon)$

6 　　执行动作 a，观察奖励 R 和下一个状态 s'

7 　　$L(\theta) \leftarrow \frac{1}{2}[R+\gamma \max_a q_\pi(s', a'; \boldsymbol{\theta}) - q_{\pi\pi}(s, a; \theta)]^2$

8 　　$\theta \leftarrow \boldsymbol{\theta} - \eta \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$

9 　　$s \leftarrow s'$

10 　**until** s 是终止状态

11 until q_π 收敛

12 $\pi(s) := \arg\max_a q(s, a)$

采样不足：由于相邻的样本来自同一条轨迹，因此样本之间相关性太强，使得集中优化相关性强的样本会导致神经网络处理其他样本无法取得较好效果。Q学习的一个特性是倾向于反复采样更新一部分状态，这部分状态来源于Q学习算法采样的轨迹，其并不是均匀分布在整个状态空间中。Q学习这个特性在使用数组记录动作-价值函数时只会导致对部分状态的价值估计越来越准确。但是在使用回归模型拟合动作-价值函数时，连续对一组相关性强的状态进行更新虽然会使模型对这些状态的价值估计越来越准确，但也会导致对那些没有被更新状态的价值估计产生更大偏差。

难以收敛：在算法6.8中第7行计算损失函数时，参数$\boldsymbol{\theta}$被同时用来计算动作-价值函数的目标值（估计值）$R+\max_{a'} q_\pi(s',a';\boldsymbol{\theta})$和当前值$q_\pi(s,a;\boldsymbol{\theta})$，因此对$\boldsymbol{\theta}$的更新会同时影响这两个值。对于当前样本的损失函数计算公式而言，令损失函数值为0的动作-价值函数取值不是唯一的，算法既可以调整当前值$q_\pi(s,a;\boldsymbol{\theta})$使其等于目标值，也可以调整目标值使其等于当前值。在单步优化目标不明确的情况下，由于样本具有概率性，这样会使得算法的收敛可能变得不稳定。

策略评估：深度Q网络（Deep Q-Network，DQN）

对于样本相关性太强的问题，DQN采用经验重现（experience replay）的方法来应对（如图6.15）。经验重现将算法探索的片段以(s,a,R,s')的四元组形式存在一张表中，每当算法探索得到一个新的四元组，便将该四元组加入经验重现表；而在算法对参数θ进行更新时，则从经验重现表中随机选取一批样本用于计算损失函数。使用经验重现主要有两个好处，一方面是样本之间的相关性显著减弱，因为随机采样得到的样本状态会更均匀地分布在已探索的样本空间中。另一方面是过去的经验可以被重复利用，因此提高了信息利用的效率。

用另一组参数$\boldsymbol{\theta}^-$来计算动作-价值函数的预测值。具体来说，对损失函数进行了如下修改：

$$L(\boldsymbol{\theta}) = \frac{1}{2}\left[R + \gamma \max_{a'} q_\pi(s', a'; \boldsymbol{\theta}^-) - q_\pi(s, a; \boldsymbol{\theta})\right]^2$$

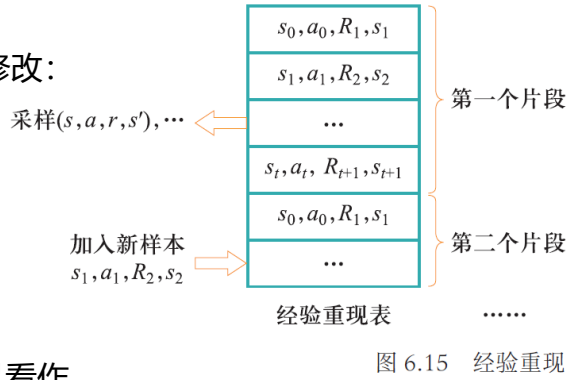


图 6.15　经验重现

参数$\boldsymbol{\theta}^-$保持相对稳定，以较低的频率更新$\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$。因此在算法更新$\theta$时，目标值可以看作一个常数，求解目标变成了一个普通的回归问题，这样使得算法更加稳定

同济大学 TONGJI UNIVERSITY

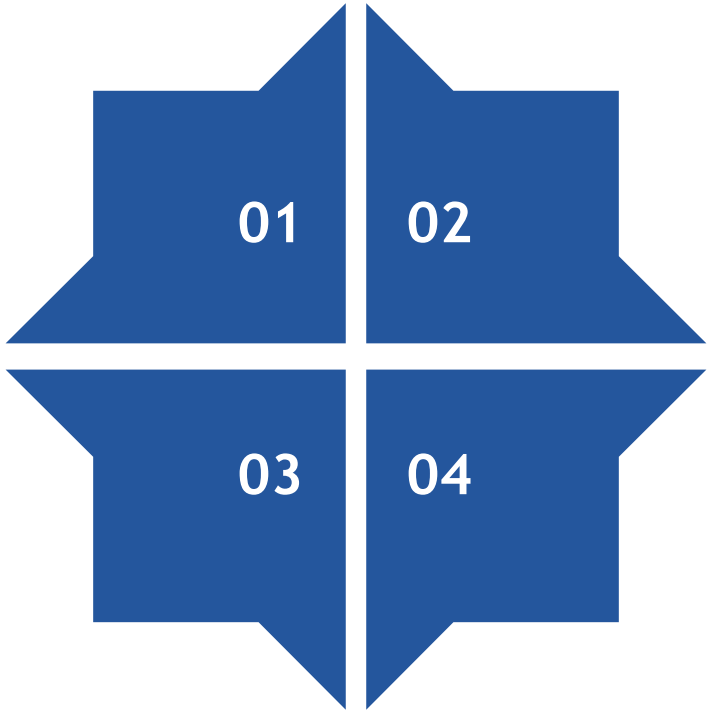
目录

基于价值的强化学习

基于策略的强化学习

深度强化学习在围棋中的应用

案例分析——AlphaGo



策略评估：策略梯度法（Policy Gradient Methods）

通过直接参数化策略函数的方法求解强化学习问题，由于算法需要求参数化的策略函数的梯度，因此这些方法称为策略梯度法（Policy Gradient Methods）

- 策略函数的参数化可以表示为$\pi_{\boldsymbol{\theta}}(s,a)$，其中$\boldsymbol{\theta}$为一组参数，函数取值表示在状态$s$下选择动作$a$的概率。和Q学习的$\epsilon$贪心策略相比，这种参数化的一个显著好处是：选择一个动作的概率是随着参数的改变而光滑变化的，实际上这种光滑性对算法收敛有更好的保证。
- 为了优化策略函数，首先需要有一个对策略函数优劣进行衡量的标准。不妨假设强化学习问题的初始状态为s_0，不难定义算法希望达到的最大化目标为

$$J(\boldsymbol{\theta}) \coloneqq V_{\pi_{\boldsymbol{\theta}}}(s_0)$$

- 这是一个关于参数$\boldsymbol{\theta}$的函数，其值为从初始状态出发的价值函数。如果能求出梯度$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$，就可以用梯度上升（Gradient Ascent）法求解这个问题了，即求解价值函数的最大值。

策略评估：策略梯度定理

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \sum_{s} \mu_{\pi_{\boldsymbol{\theta}}}(s) \sum_{a} q_{\pi_{\boldsymbol{\theta}}}(s,a) \pi_{\boldsymbol{\theta}}(s,a)$$

$$\propto \sum_{s} \mu_{\pi_{\boldsymbol{\theta}}}(s) \sum_{a} q_{\pi_{\boldsymbol{\theta}}}(s,a) \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(s,a)$$

$\mu_{\pi_{\boldsymbol{\theta}}}(s)$称为策略$\pi_{\boldsymbol{\theta}}$的策略分布（这里假设折扣系数$\gamma = 1$）。在持续问题中，$\mu_{\pi_{\boldsymbol{\theta}}}(s)$为算法在策略$\pi_{\boldsymbol{\theta}}$安排下从$s_0$出发经过无限多步后位于状态$s$的概率；在分段问题中，$\mu_{\pi_{\boldsymbol{\theta}}}(s)$为归一化后的算法从$s_0$出发访问$s$次数的期望。当$\gamma \in (0,1)$时，则需要给每个状态的$\mu_{\pi_{\boldsymbol{\theta}}}(s)$值加上一个权重。在保证算法通用性的前提下，为了简化说明，下文在进行公式推导时始终假设$\gamma = 1$，但是在算法流程中会体现折扣系数γ的影响。

上式称为**策略梯度定理（Policy Gradient Theorem）**，它提供了策略梯度法的求解思路：如果能够计算或估计策略函数的梯度，智能体就能直接对策略函数进行优化。

策略评估：基于蒙特卡洛采样的策略梯度法

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \propto \sum_s \mu_{\pi_{\boldsymbol{\theta}}}(s) \sum_a q_{\pi_{\boldsymbol{\theta}}}(s,a) \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(s,a)$$

$$= \sum_s \mu_{\pi_{\boldsymbol{\theta}}}(s) \sum_a \pi_{\boldsymbol{\theta}}(s,a) \left[q_{\pi_{\boldsymbol{\theta}}}(s,a) \frac{\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(s,a)}{\pi_{\boldsymbol{\theta}}(s,a)} \right]$$

$$= \mathbb{E}_{s,a\sim\pi} \left[q_{\pi_{\boldsymbol{\theta}}}(s,a) \frac{\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(s,a)}{\pi_{\boldsymbol{\theta}}(s,a)} \right]$$

$$= \mathbb{E}_{s,a\sim\pi} \left[q_{\pi_{\boldsymbol{\theta}}}(s,a) \nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}(s,a) \right]$$

上述推导中利用了$\nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}(s,a) = \frac{\mathrm{Ine} \times \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(s,a)}{\pi_{\boldsymbol{\theta}}(s,a)} = \frac{\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(s,a)}{\pi_{\boldsymbol{\theta}}(s,a)}$

由于$q_{\pi_{\boldsymbol{\theta}}}(s,a) = \mathbb{E}_{\pi}[G_t|s,a]$，因此存在可变形如下：

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \propto \mathbb{E}_{s,a\sim\pi} \left[\mathbb{E}_{\mathcal{T}(s,a)\sim\pi} [G_t|s,a] \nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}(s,a) \right]$$

$$= \mathbb{E}_{s,a,\mathcal{T}(s,a)\sim\pi} [G_t \nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}(s,a)]$$

其中$\mathcal{T}(s,a)$表示从状态s开始执行动作a得到的一条轨迹（不包括s和a），G_t为从状态s开始沿着轨迹$\mathcal{T}(s,a)$运动所得回报。可以使用蒙特卡洛采样法来求解，即算法只需根据策略来采样一个状态s、一个动作a和将来的轨迹，就能构造公式中求取期望所对应的一个样本。

策略评估：基于蒙特卡洛采样的策略梯度法

上述估计策略梯度的算法称为REINFORCE，其流程如算法6.9所示。在这个算法中，通过利用采样得到的轨迹片段来估计梯度，并使用梯度上升法来优化策略。注意算法6.9中第6行的更新公式中多了一个γ^t，这个差别正是由前文所说的公式推导时假设$\gamma = 1$导致的。在实现中，这个γ^t常常会被省略。另一个需要注意的地方是算法6.9的第6行中参数与梯度相加而非减去梯度，这是由于算法的目标是最大化$J(\boldsymbol{\theta})$而不是经常所采用的最小化该函数的方法。

算法6.9通过执行完一个完整片段所得实际反馈值来估计策略梯度，这一点和策略评估的蒙特卡洛法很相似，两个算法都要求问题是分段的，且只有在一个片段结束后算法才能进行更新操作。

算法 6.9 REINFORCE 算法

函数： REINFORCE

输入： 马尔可夫决策过程 MDP $=(S, A, P, R, r)$

输出： 策略 π

1 随机初始化 $\boldsymbol{\theta}$

2 **repeat**

3 　根据策略 $\pi_{\boldsymbol{\theta}}$ 采样一个片段 $s_0, a_0, R_1, s_1, \cdots, s_{T-1}, a_{T-1}, R_T$

4 　**for** $t \leftarrow 0$ to $T-1$ **do**

5 　　$G \leftarrow \sum_{k=1}^{T-t} \gamma^{k-1} R_{t+k}$

6 　　$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \gamma^t G \nabla_{\boldsymbol{\theta}} \ln \pi_{\boldsymbol{\theta}}(s_t, a_t)$

7 　**end**

8 **until** $\boldsymbol{\theta}$ 收敛

策略评估： Actor-Critic算法

- 是否能够从时序差分的角度设计一个策略梯度求解方法呢？时序差分与蒙特卡洛采样的核心差别在于使用下一时刻状态的价值函数来估计当前状态的价值函数，而不是使用整个片段的反馈值，体现在算法中即使用$R+\gamma V_{\pi_\theta}(s')$来代替算法6.9中的变量$G$。算法6.10给出了基于时序差分的策略梯度法的流程，其名称为Actor-Critic算法。
- 在算法6.10中，第9行用$R+\gamma V_{\boldsymbol{w}}(s')$作为$q_{\pi_\theta}(s,a)$的估计值，而非当前片段的实际回报。第8行中仍然用价值函数估计值和当前值之间误差的平方作为损失函数，但值得注意的一点是，算法假设$\nabla_{\boldsymbol{w}} V_{\boldsymbol{w}}(s')=0$，即假设目标值中的$V_{\boldsymbol{w}}(s')$是一个常数，这与Q学习中目标网络的做法不谋而合。
- Actor-Critic算法最初的研究出现时，人们对策略梯度法还没有形成体系化认识。研究者们认为参数化的策略函数就像一个执行者（actor），负责动作的选择，而参数化的价值函数就像一个评论家（critic），评价执行者选出的动作的好坏，并成为执行者更新策略的依据，Actor-Critic算法由此得名。

算法 6.10 Actor−Critic 算法

函数： ActorCritic

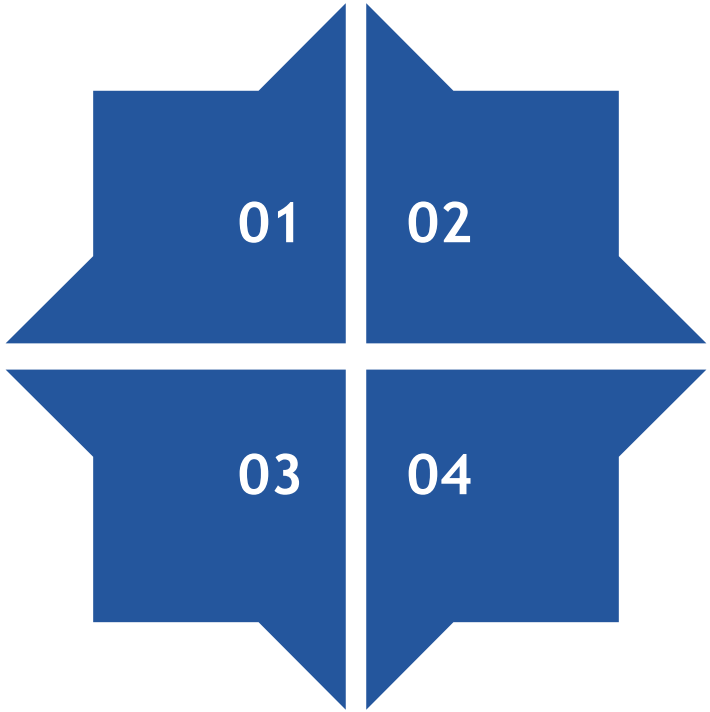
输入： 马尔可夫决策过程 MDP $=(S, A, P, R, \gamma)$

输出： 策略 π

1 随机初始化 θ, w
2 **repeat**
3 　$s \leftarrow$ 初始状态
4 　$t \leftarrow 0$
5 　**repeat**
6 　　$a \sim \pi_\theta(s, .)$
7 　　执行动作 a，观察奖励 R 和下一时刻状态 s'
8 　　$w \leftarrow w+\eta_w \gamma^t[R+\gamma V_w(s')-V_w(s)] \nabla_w V_w(s)$
9 　　$\theta \leftarrow \theta+\eta_\theta \gamma^t[R+\gamma V_w(s')] \nabla_\theta \ln \pi_\theta(s, a)$
10 　　$t \leftarrow t+1$
11 　**until** s 是终止状态
12 **until** θ 收敛

同济大学 TONGJI UNIVERSITY

目录



基于价值的强化学习

基于策略的强化学习

深度强化学习在围棋中的应用

案例分析——AlphaGo

深度强化学习在围棋游戏中的应用

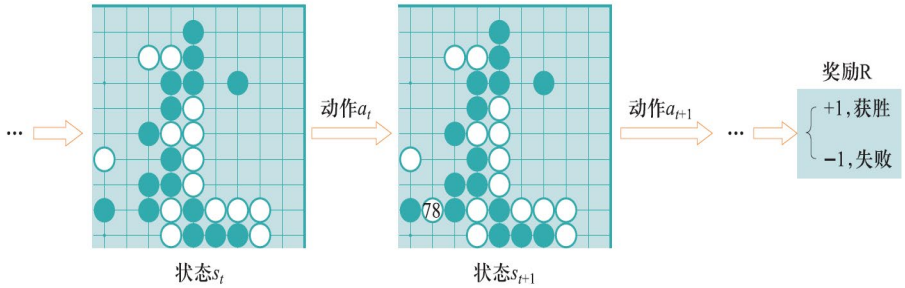


图 6.17　围棋游戏一个片段的轨迹

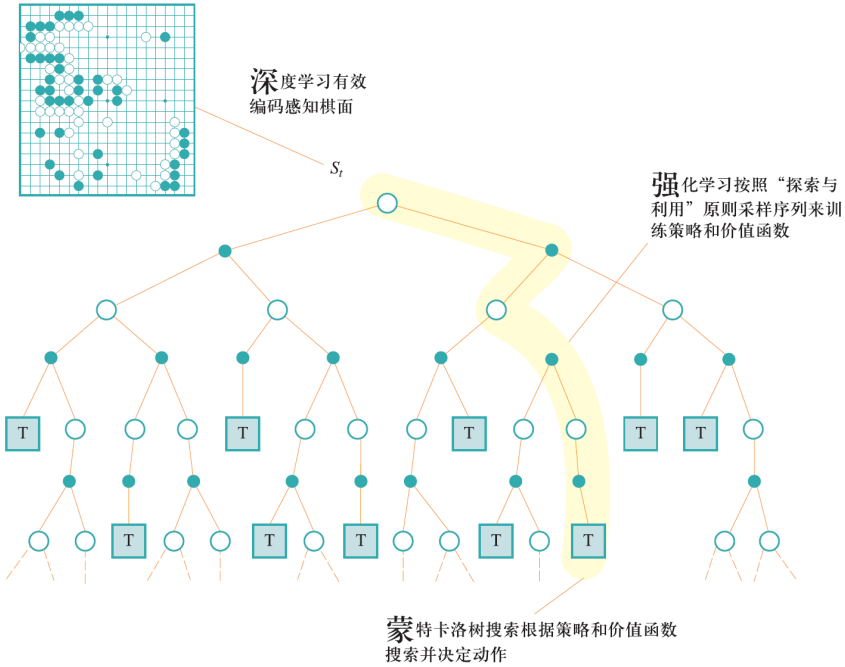


图 6.18　AlphaGo 算法中三个重要的人工智能算法

蒙特卡洛树搜索（MCTS）

普通的基于价值的强化学习选择Q值最大的动作作为最优动作，而基于**UCB**（Upper Confidence Bound，置信上界）的动作选择不仅考虑动作的Q值，还考虑动作已经被选择的次数，并优先选择之前被选择较少的动作： $a = \text{argmax}_{a'}(q_\pi(s,a') + c\sqrt{\frac{\ln N}{N(s,a')}})$。对于AlphaGo，$\text{UCB}(s,a) = Q(s,a) + c\frac{P(s,a)}{1+N(s,a)}$。其中，$P(s,a)$ 表示 (s,a) 出现的先验概率，由监督学习策略网络 p_σ 计算得出，其网络参数是使用接近3000万个人工标注的专家数据进行有监督学习得到的。

Rollout通过平均从每个可能的动作开始并随后遵循给定策略的许多模拟轨迹的回报，来估计给定策略的动作值。当动作值估计被认为足够准确时，就会执行具有最高估计值的动作（或其中一个动作），然后从产生的下一个状态重新开始该过程。Rollout的目标并非为给定策略 π 估计完整的最优动作价值函数 q* 或完整的动作价值函数 q_π，而是仅针对当前状态和给定策略生成动作价值的蒙特卡罗估计值，无需为每个状态 - 动作对采样结果，也无需在状态空间或状态 - 动作空间上近似函数。相比 p_σ，AlphaGo中的Rollout策略（也称为快速走子或随机模拟）是在800万个实际对局数据上用有监督学习方法训练得到的简单的线性网络，该网络的每个线程在一秒内可以模拟 1000次完整的对局。

蒙特卡洛树搜索（MCTS）

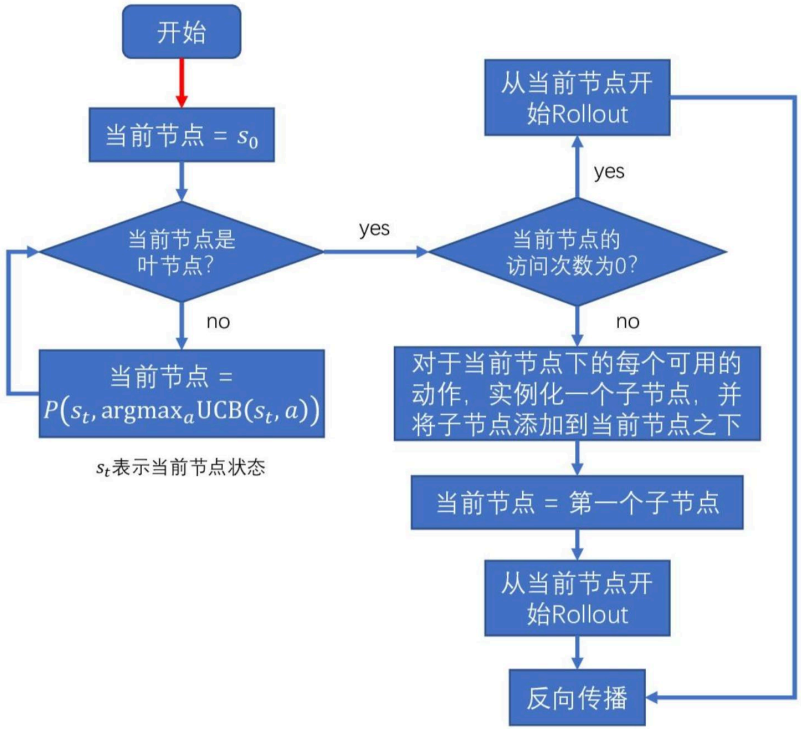
基本版本的MCTS的每次迭代都包含以下四个步骤：

1.动作选择（Selection）。从根节点开始，基于附加在树的边上的动作值的树策略遍历树以选择一个叶节点。

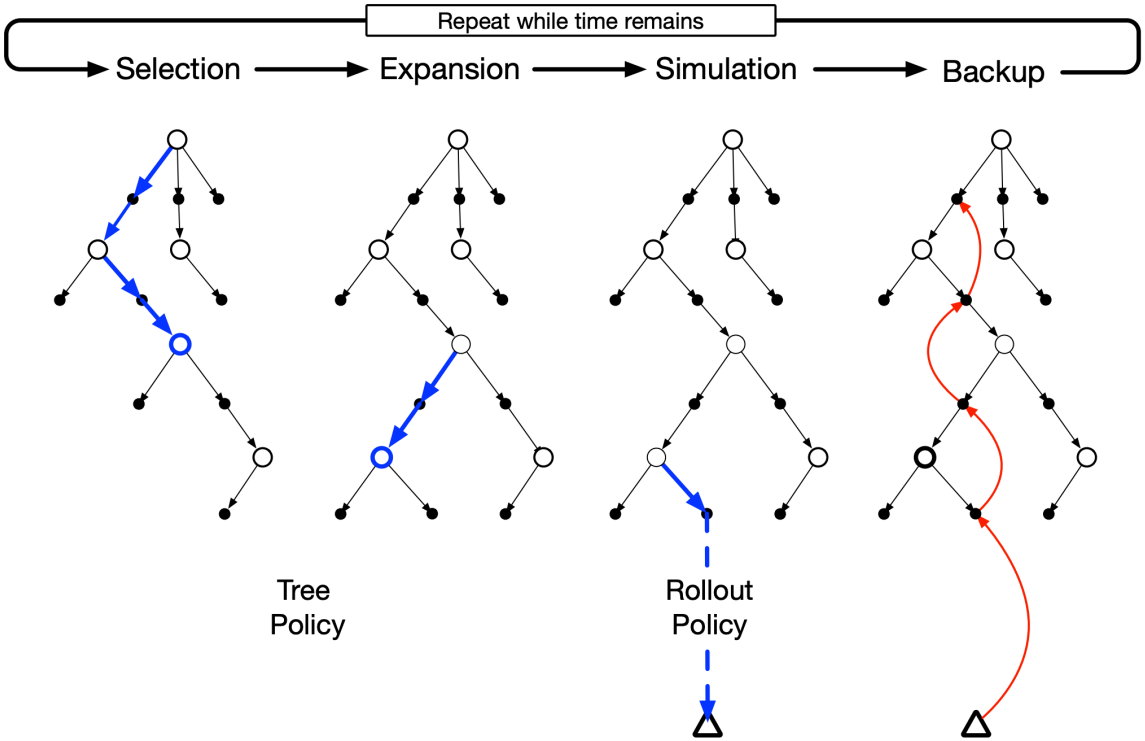
2.节点扩展（Expansion）。在某些迭代中，从所选的叶节点开始扩展树，通过未探索的动作从所选节点添加一个或多个子节点。

3.预演（Rollout）。从所选节点开始，或者从其新添加的子节点之一（如果有的话）开始，使用rollout策略选择动作来运行一个完整回合的模拟。

4.反向传播（Backpropagation）。模拟回合所产生的回报会被回溯，用于更新或初始化在本次迭代中由树策略遍历的树的边所关联的动作值。 Rollout 策略访问的状态和动作不会在搜索树之外保存任何值，因为 MCTS 只关注搜索树中的节点。



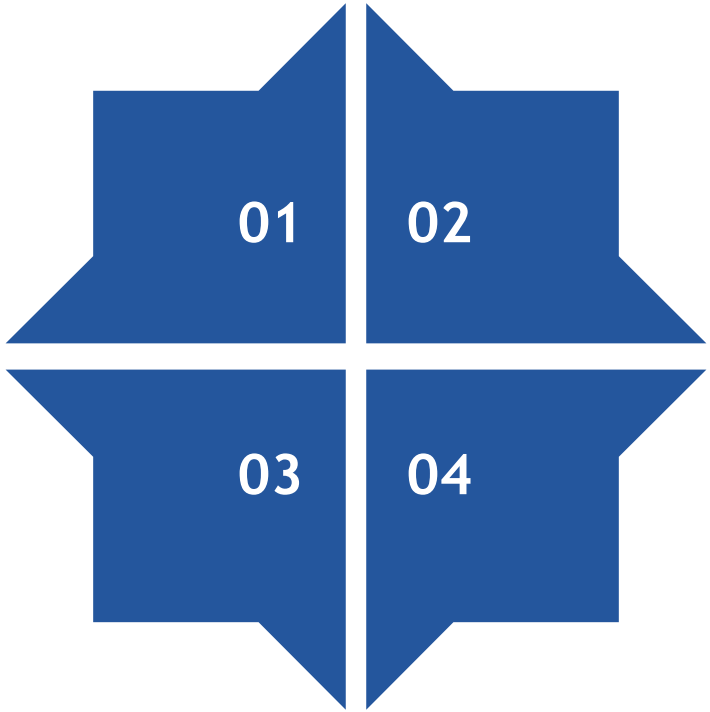
蒙特卡洛树搜索（MCTS）



同济大学 TONGJI UNIVERSITY

目录

基于价值的强化学习



基于策略的强化学习

深度强化学习在围棋中的应用

案例分析——AlphaGo

案例分析——AlphaGo

问题描述：
2016年，AlphaGo在人机围棋大战中连胜世界围棋冠军李世石3局，成为人工智能里程碑的事件。
2017 年，AlphaGo 打败了世界排名第一围棋手柯洁。



AlphaGo如何实现下棋?

借鉴专业围棋棋手的思维

“走一步、看三步”

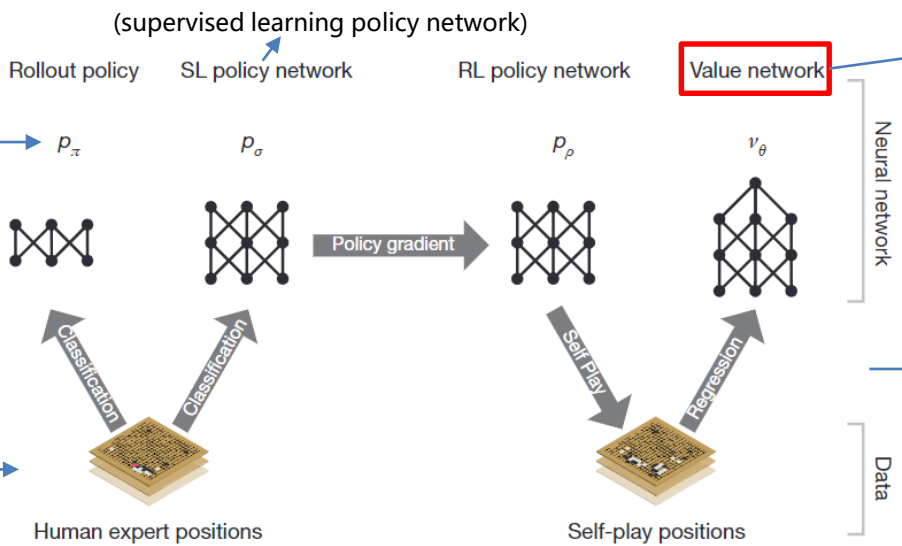
- 分析全局和局部形势
- 判断棋局，选出几个可能的落子点
- 预测接下来几步棋局的变化，判断最佳落子点

AlphaGo 的主要创新点在于用改进的MCTS（异步策略和价值 MCTS，简称APV-MCTS）进行走子，这个改进版本的MCTS使用强化学习得到的策略和价值函数做指导，其中函数逼近使用了深度卷积神经网络，且该神经网络参数用人工标记的对局数据进行预训练，而不是随机初始化网络参数。

案例分析——AlphaGo

问题分析：

1. 选点（这个局面走哪几个地方比较强？）



输出：预测人类专家下一步走棋位置

输入：当前棋局

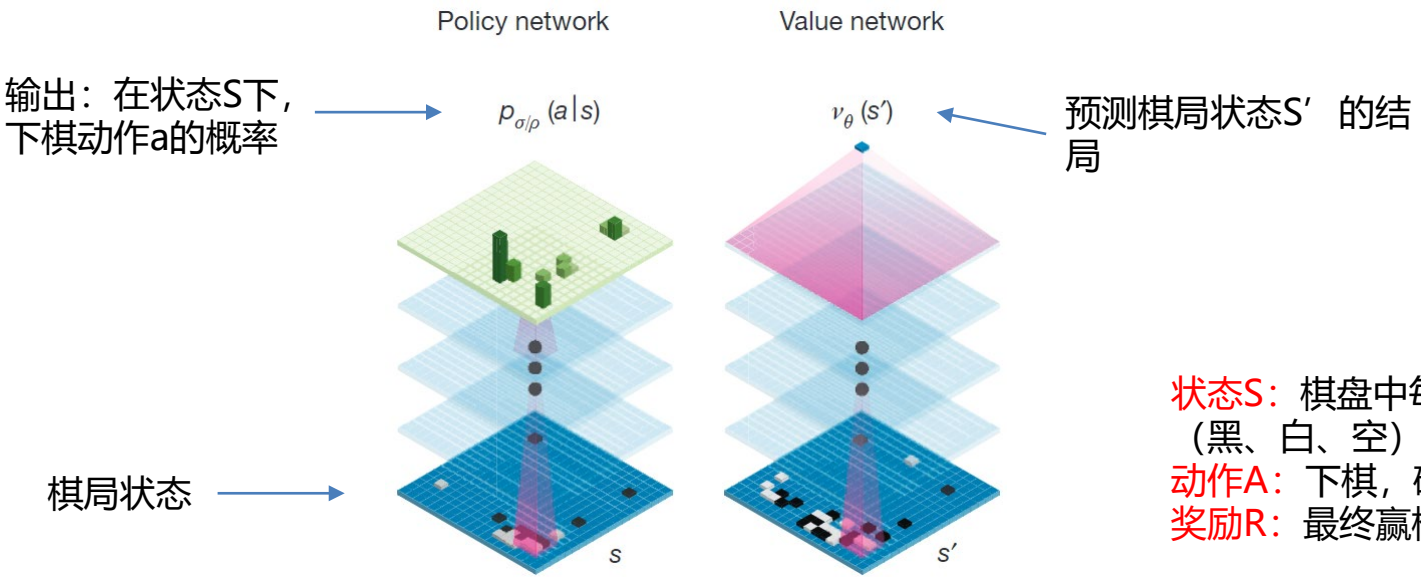
对每一个可能的落子点一个分数，选择分数最大的那个落子点

为了提高策略的质量，通过自我对弈进行强化学习，其中对局双方中的待更新的网络使用当前的强化学习策略网络，对手使用的是从早期迭代得到的版本中随机选择的策略

如果当前策略赢，收益信号就是+1，如果输就是-1，其他情况就是 0。
这些对局过程是直接依据两个策略网络进行落子的，没有涉及 MCST。

案例分析——AlphaGo

2.预测接下来几步棋局的变化



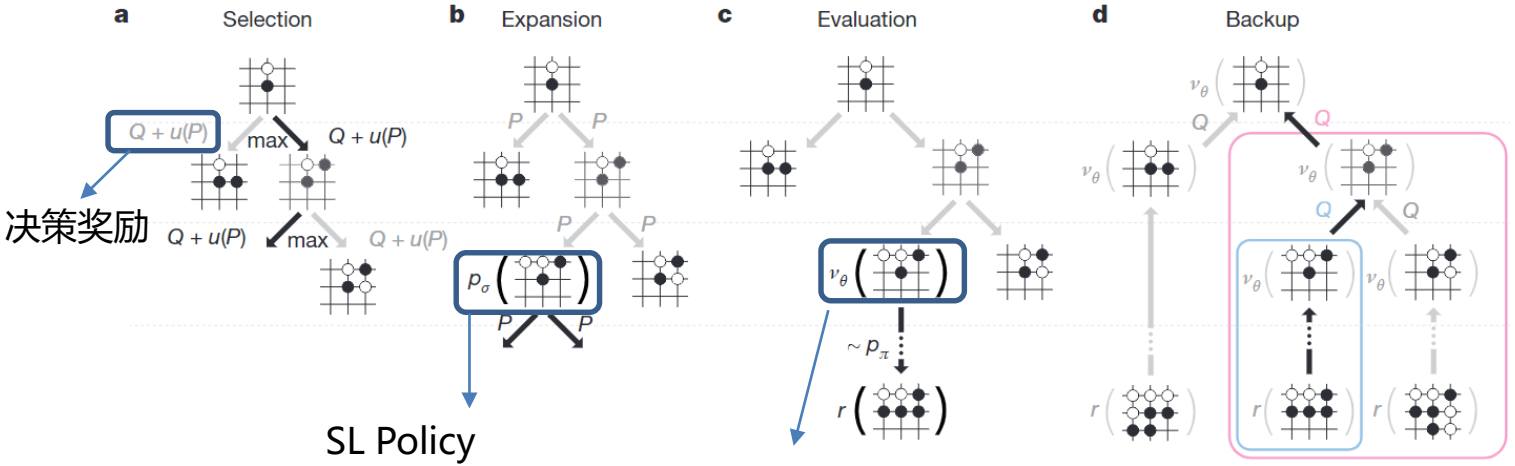
状态S：棋盘中每个落子点的状态（黑、白、空）
动作A：下棋，确定落子点位置
奖励R：最终赢棋

基础 MCTS只用预演的回报值来估计新增状态节点的价值，而在APV-MCTS 中对新增状态节点的估计来源于两个部分：预演估计值和强化学习方法得到的价值函数。

案例分析——AlphaGo

3. 判断最佳落子点

利用蒙特卡洛搜索树（Monte Carlo Tree Search, MCTS）将以上两个步骤联系起来，判断最佳落子策略。策略网络用于缩小搜索空间，给出可能的好棋步，而价值网络用于评估在各种棋步后的棋局状态。



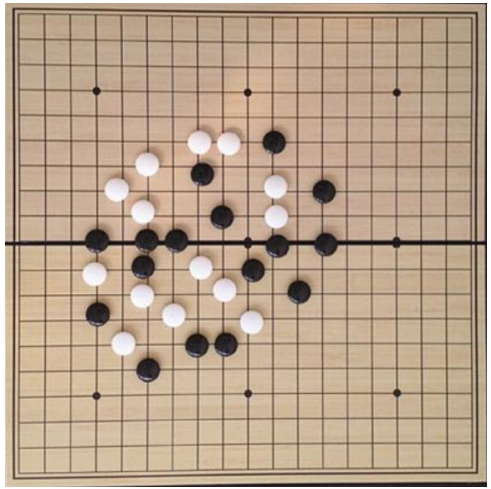
案例分析——AlphaGo Zero

问题背景：

为简化AlphaGo的算法，2017年Google的Deep Mind团队提出了AlphaGo Zero——完全不依赖人类下棋经验的自学习算法。

问题描述：

给定任意一个状态，寻找最优应对策略，使得最终可以获得棋盘上的最大地盘。



AlphaGo Zero 是如何简化算法，实现下棋?

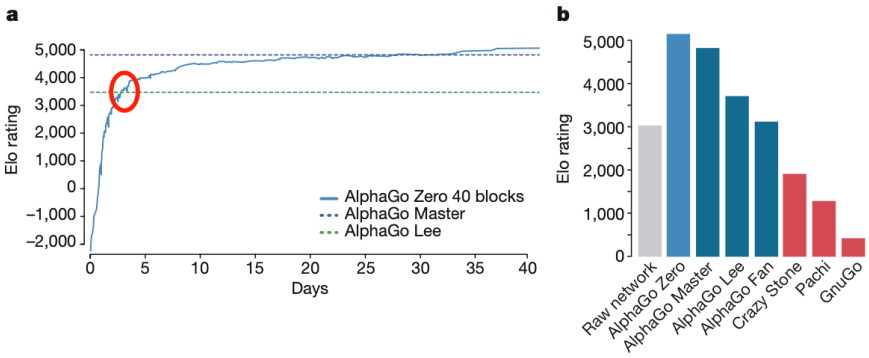
AlphaGo Zero抛弃了人类棋谱，完全通过自对弈来学会下棋

整个过程可以分为以下几个部分：

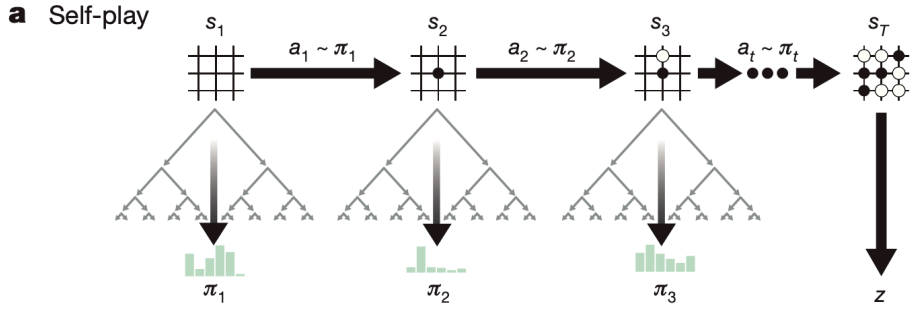
- 自学习阶段
- 神经网络训练阶段

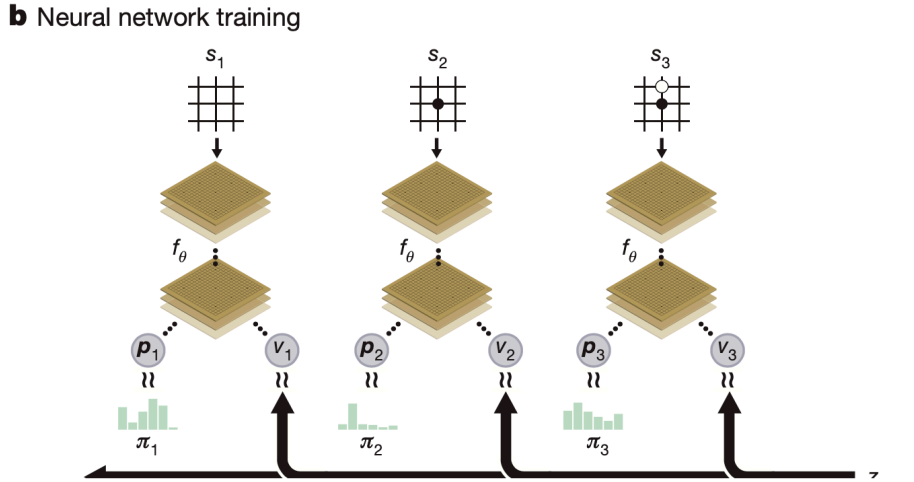
案例分析——AlphaGo Zero

- 通过两个智能体自我对弈收集大量对弈数据与胜负信息
- 使用强化学习训练神经网络用于对弈策略
- 完全不需要任何人类知识或棋谱
- 智能体通过强化学习从自我对弈中不断学习



仅训练三天就超过了依赖大量人类棋谱的李世石版本

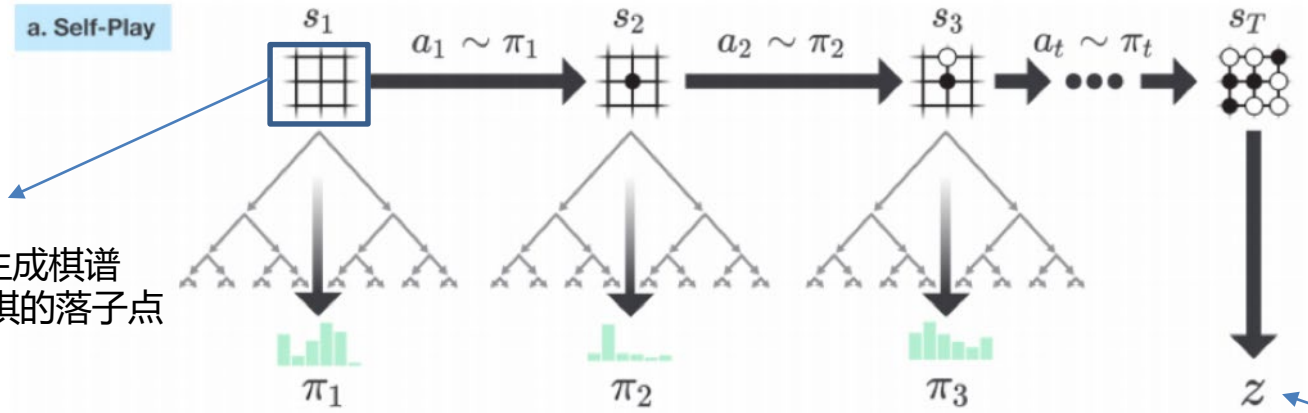




案例分析——AlphaGo Zero

1.自学习阶段

首先生成19*19=361个交叉点的棋谱，将棋谱作为训练神经网络的输入，训练好的神经网络用来预测落子位置和胜率。



利用MCTS生成棋谱
确定每一步棋的落子点

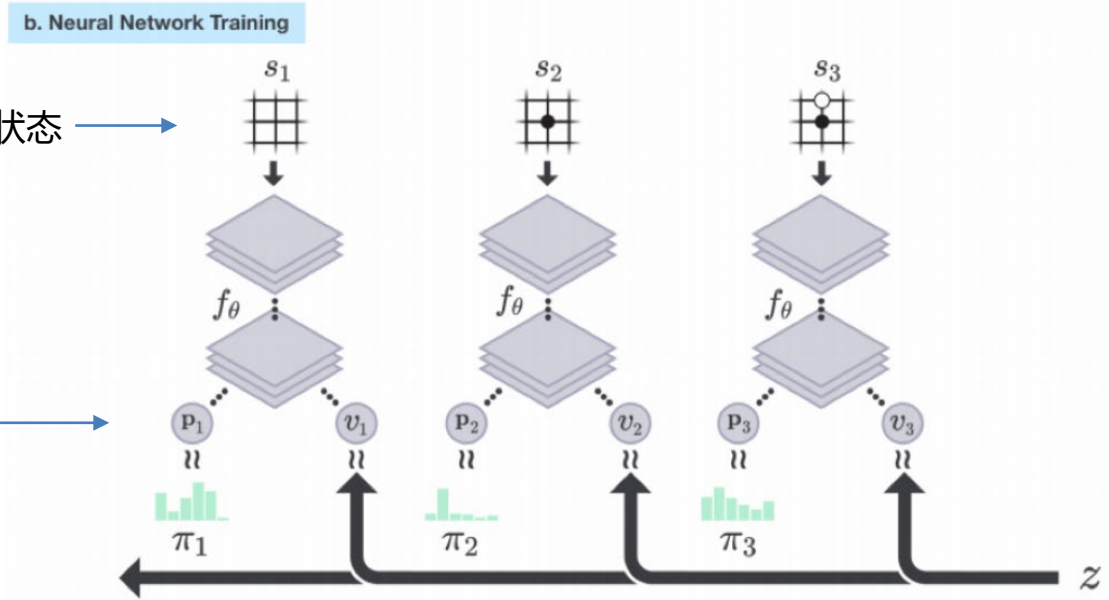
胜负结果

案例分析——AlphaGo Zero

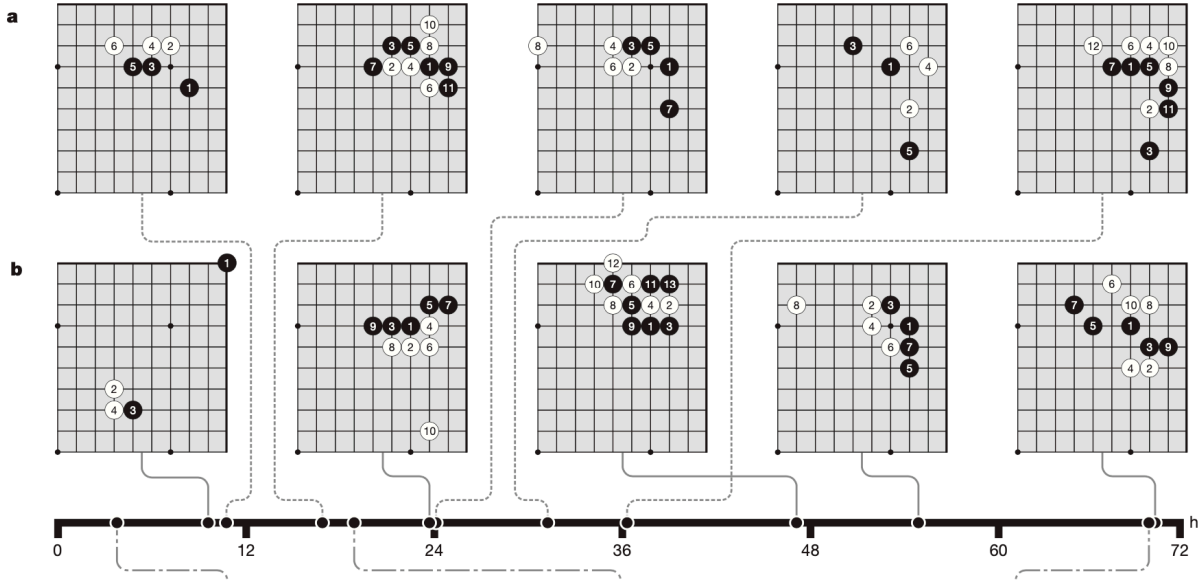
2.神经网络训练

当前棋局状态

输出：预测的落子的概率P和胜率v



案例分析——AlphaGo Zero



- a：在自我对弈的强化学习训练中，智能体仅在不到72小时内就自己总结出来人类几百年来总结的定式下法
- b：随着训练的进行，智能体也总结出了新的AI定式，这些定式已经成为了目前围棋职业选手的首选

本章小结

- 本章介绍了基于价值的强化学习算法和基于策略的强化学习算法，这些算法都可以认为是基于通用策略迭代框架的方法，即算法是**由策略评估和策略优化两个步骤**组成的。
- 强化学习可以分为两类：依靠对环境（即马尔可夫随机过程）的先验知识或建模的算法称为**基于模型（Model-based）**的方法，反之称为无模型的方法（Model-free）；只对价值函数建模并利用策略优化定理求解的方法称为**基于价值（Value-based）**的方法，对策略函数建模并利用策略梯度定理求解的方法称为基于策略（Policy-based）的方法。